香港科技大學(廣州)
THE HONG KONG UNIVERSITY OF SCIENCE AND TECHNOLOGY (GUANGZHOU)

Machine Learning Basis

Instructor: Hao Wang

AI Thrust, Information Hub



Credit: A large portion of the slides are modified from CS231n in Standford

Recaps of last stage



Filters



Example: box filter

$$g[\cdot,\cdot]$$

$$\frac{1}{9}\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

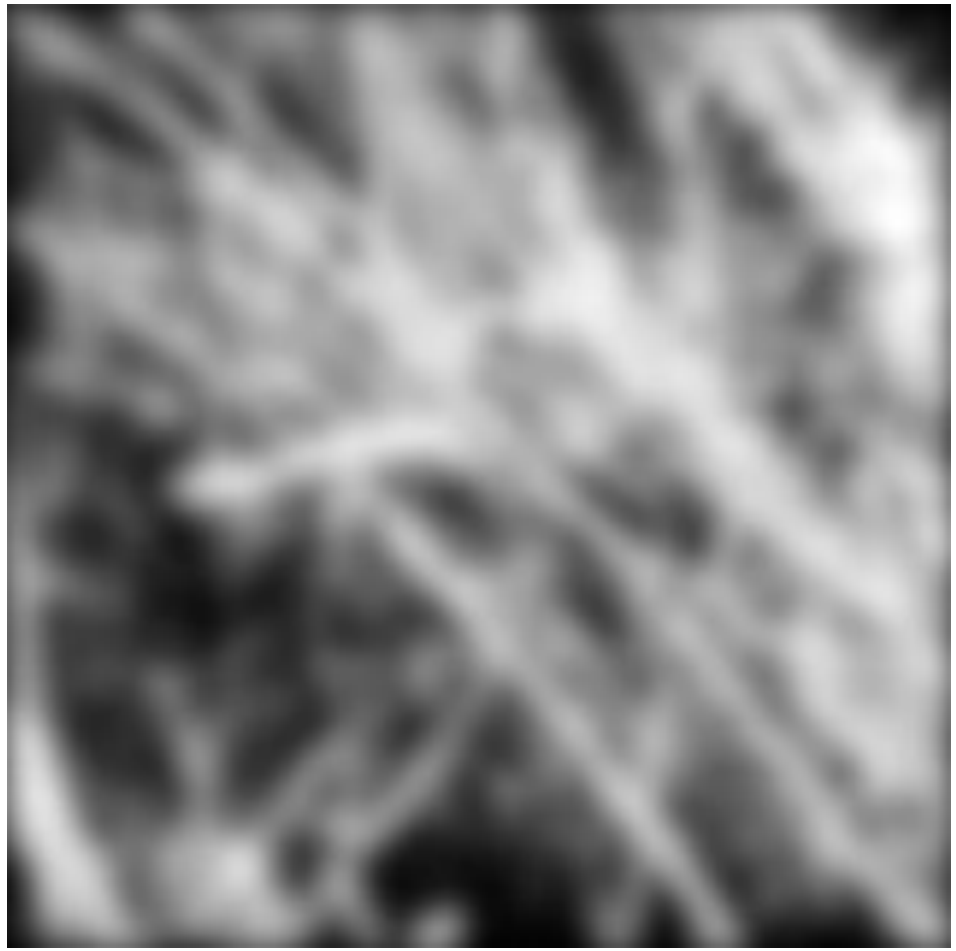


Slide credit: David Lowe (UBC)

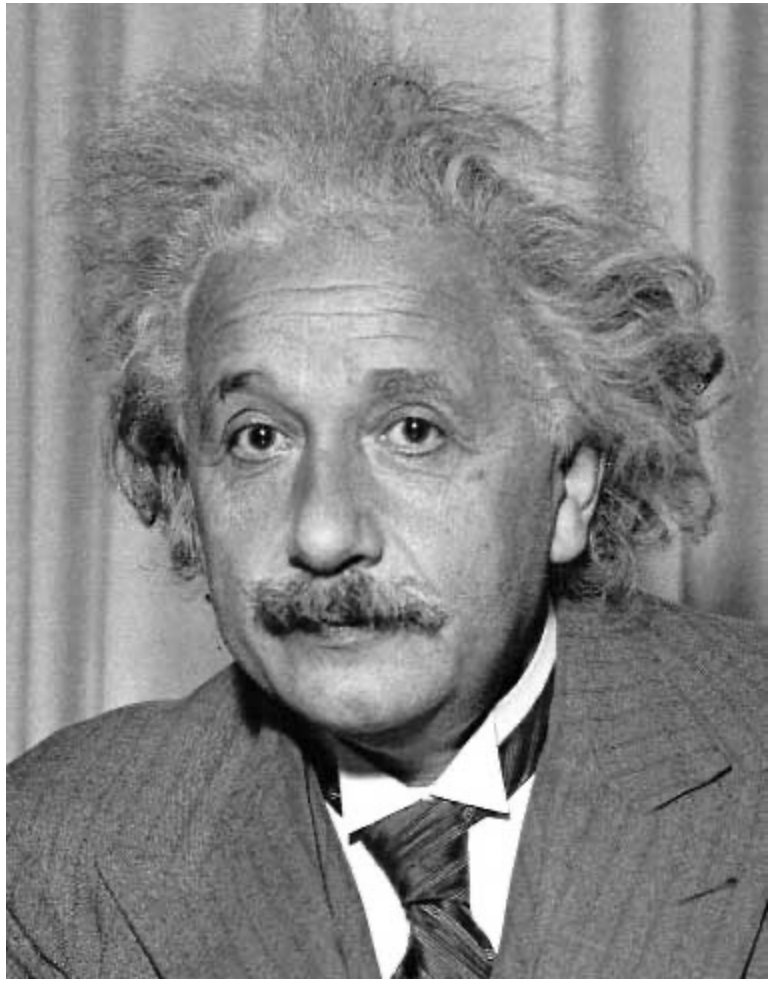
Smoothing with Gaussian filter





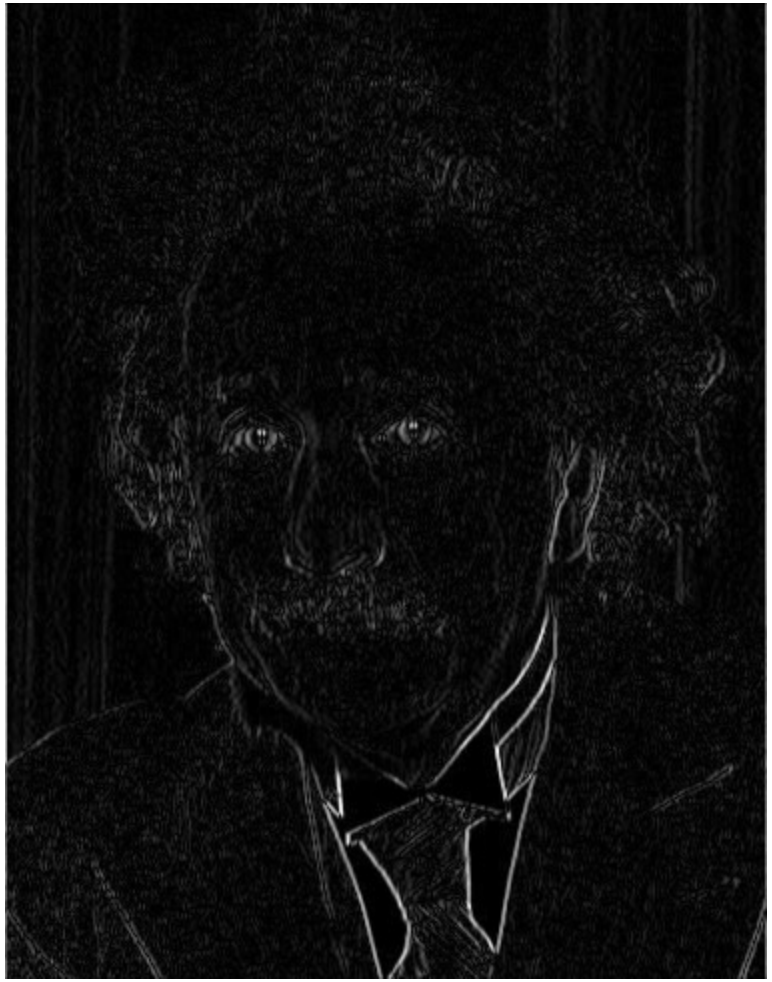


Other filters



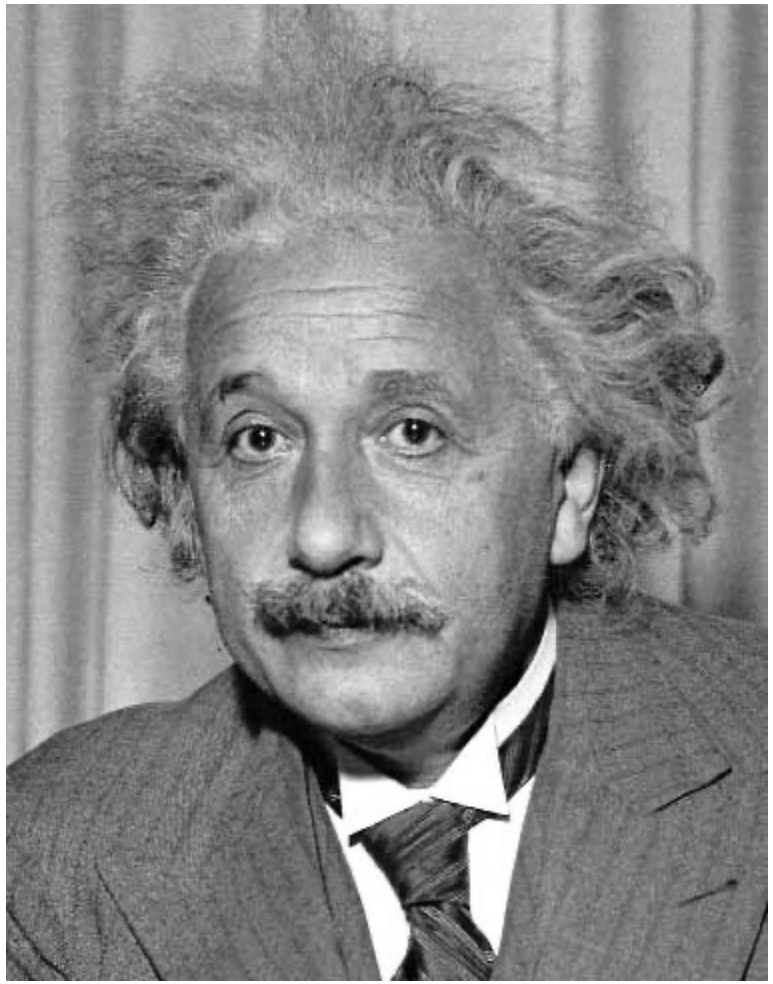
1	0	-1
2	0	-2
1	0	-1

Sobel



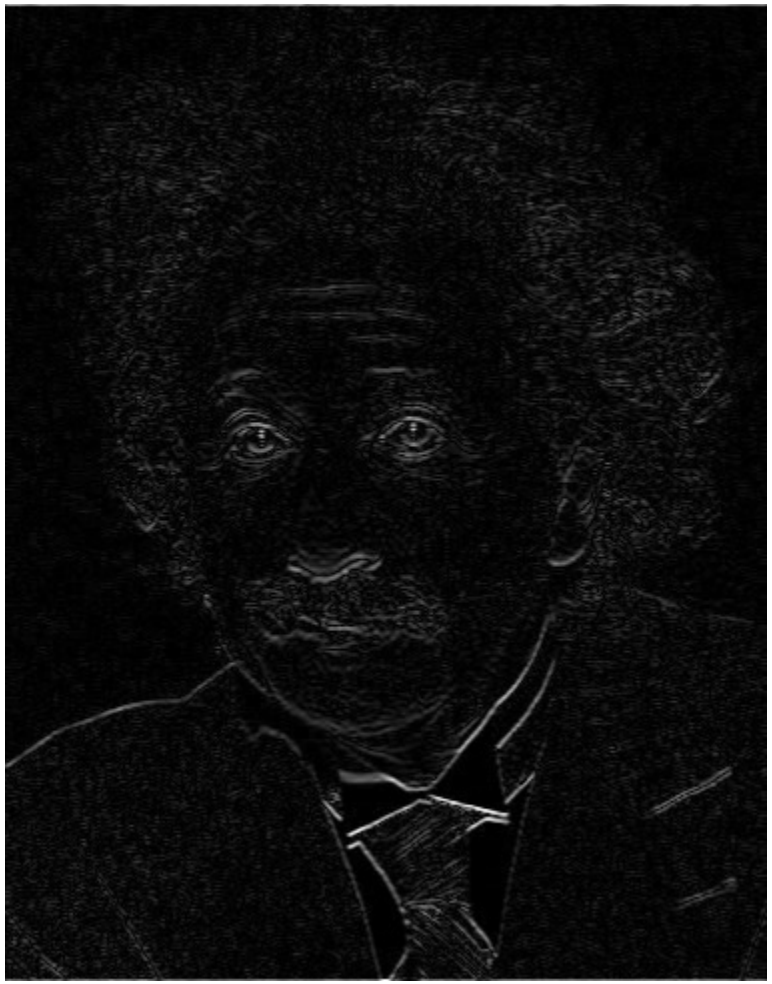
Vertical Edge
(absolute value)

Other filters



1	2	1
0	0	0
-1	-2	-1

Sobel

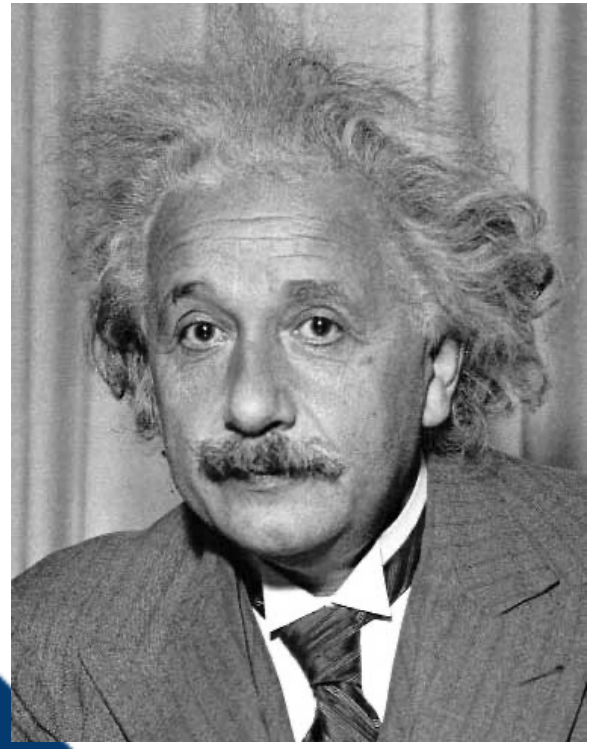


Horizontal Edge
(absolute value)

Matching with filters

- Goal: find  in image
- Method 2: Sum of Square Distance (SSD)

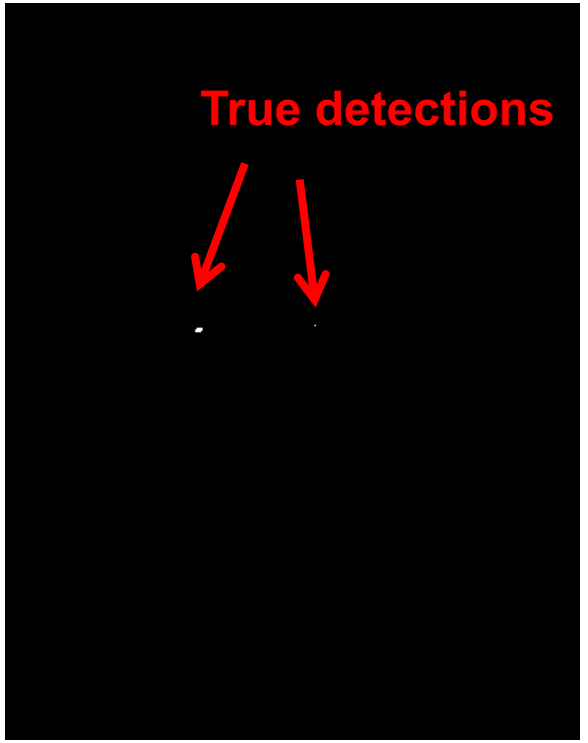
$$h[m,n] = \sum_{k,l} (f[k,l] - \ im[m+k, n+l])^2$$



Input



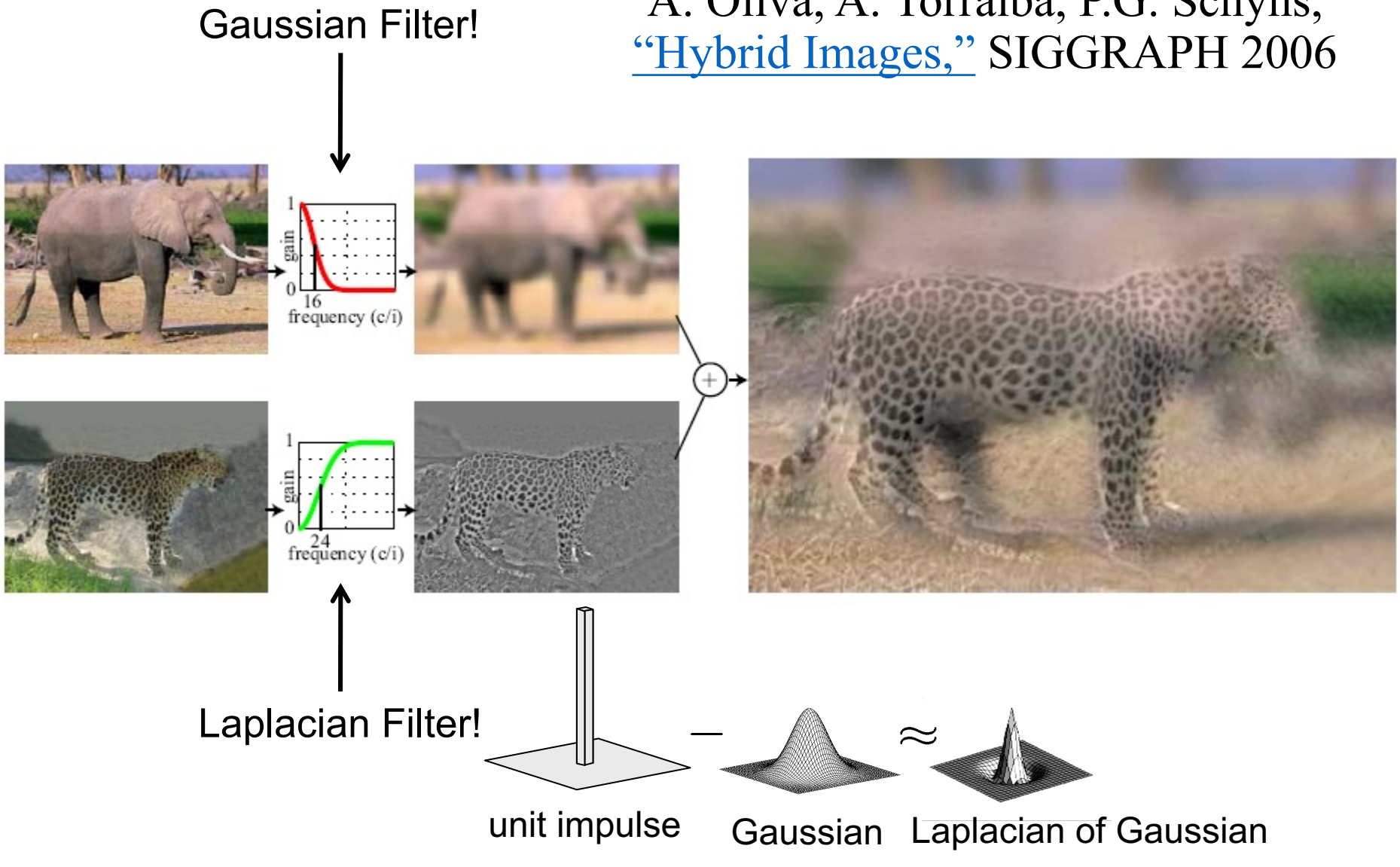
1- sqrt(SSD)



Thresholded Image

Application: Hybrid Images

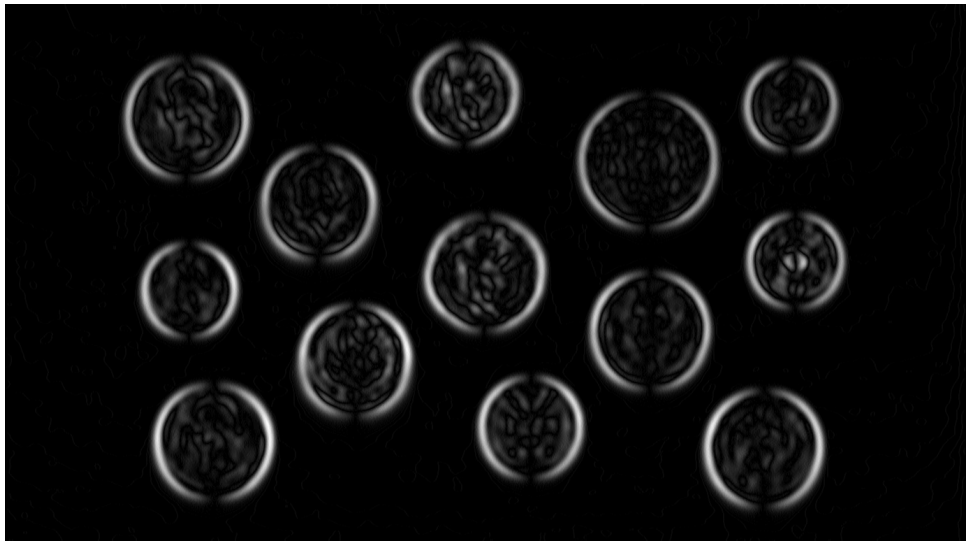
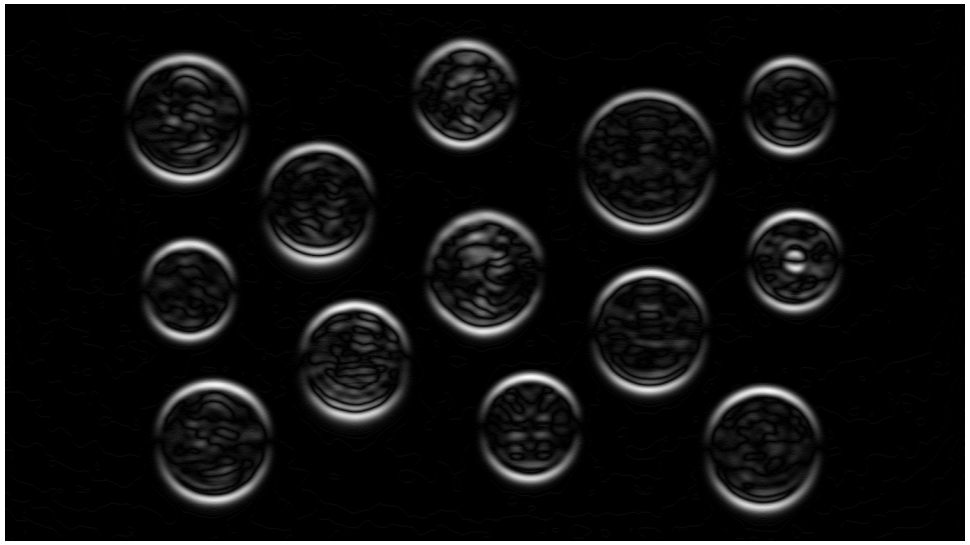
A. Oliva, A. Torralba, P.G. Schyns, "Hybrid Images," SIGGRAPH 2006



Edges and Contours



Sobel X and Y Results

Sobel X (G_x) — Vertical edges	Sobel Y (G_y) — Horizontal edges
	
Brightness changes left → right	Brightness changes top → bottom

Canny Edge Detection

The Gold Standard — 40 Years and Still Counting



Five-Step Pipeline — Overview

Step	Operation	Purpose	Depth
1	Gaussian Smoothing	Remove noise	Brief (Week 2)
2	Gradient Computation	Find candidate edges	Brief (just learned Sobel)
3	**Non-Maximum Suppression**	**Thin edges to single pixel**	**Detailed**
4	Dual Threshold	Classify strong vs weak edges	Concept
5	Edge Tracking by Hysteresis	Connect valid weak edges	Concept

> Image → Smooth → Gradient → NMS → Threshold → Track → Edges



Features and Matching



The matching Challenge

- Your brain finds matching structures (windows, corners, signs)
- Computer vision equivalent: find **correspondences**

$$(x_i, y_i) \leftrightarrow (x_i', y_i')$$

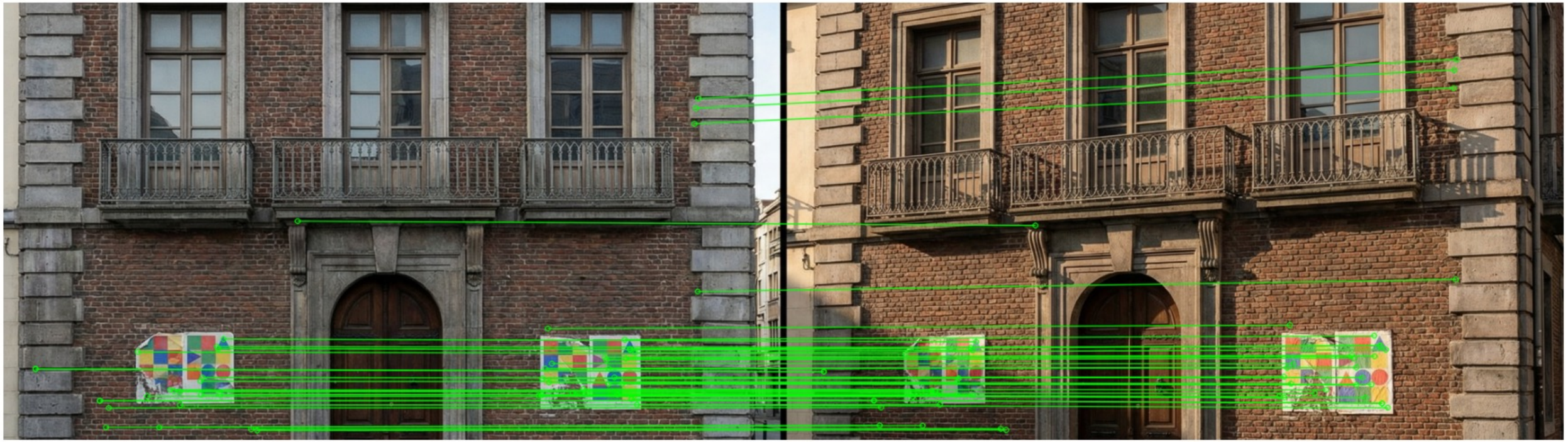
Input: Two views of the same scene.

Core Problem:
- Pixels change (Intensity, Color).
- Geometry changes (Perspective).

Goal:
Find Invariants —anchors that persist across these changes.



Finding Correspondences



Corner Detection: Harris

How Do We Define a Corner, Mathematically?



From Window Shift to a Math Quantity

Slide window W around point (x, y), shift by (u, v), measure intensity change:

$$E(u,v) = \sum_{(i,j)\in W} \left[I(i+u, j+v) - I(i,j)\right]^2$$

- Small $E \rightarrow$ little change in any direction (flat)
- Large $E \rightarrow$ significant change (edge or corner)



Derivation: First-Order Taylor Expansion

$$E(u,v) = \sum_{(i,j)\in W} [I(i+u, j+v) - I(i,j)]^2$$

For small (u, v), expand $I(i+u, j+v)$ around (i, j):

$$I(i+u, j+v) \approx I(i,j) + I_x\, u + I_y\, v$$

where $I_x = \frac{\partial I}{\partial x}$, $I_y = \frac{\partial I}{\partial y}$ (image gradients)

Intensity difference:

$$I(i+u, j+v) - I(i,j) \approx I_x\, u + I_y\, v$$

Substituting into $E(u, v)$ and rearranging into matrix form:

$$E(u,v) \approx \begin{bmatrix} u & v \end{bmatrix} \underbrace{\sum_W \begin{bmatrix} I_x^2 & I_x I_y \\ I_x I_y & I_y^2 \end{bmatrix}}_{M} \begin{bmatrix} u \\ v \end{bmatrix}$$

M = **Structure Tensor** — captures gradient energy and correlation within the window

Harris Response Function R

Key insight: $\det(M)$ and $\mathrm{trace}(M)$ equal the product and sum of eigenvalues:

$$\det(M) = \lambda_1 \lambda_2, \qquad \mathrm{trace}(M) = \lambda_1 + \lambda_2$$

Harris response:

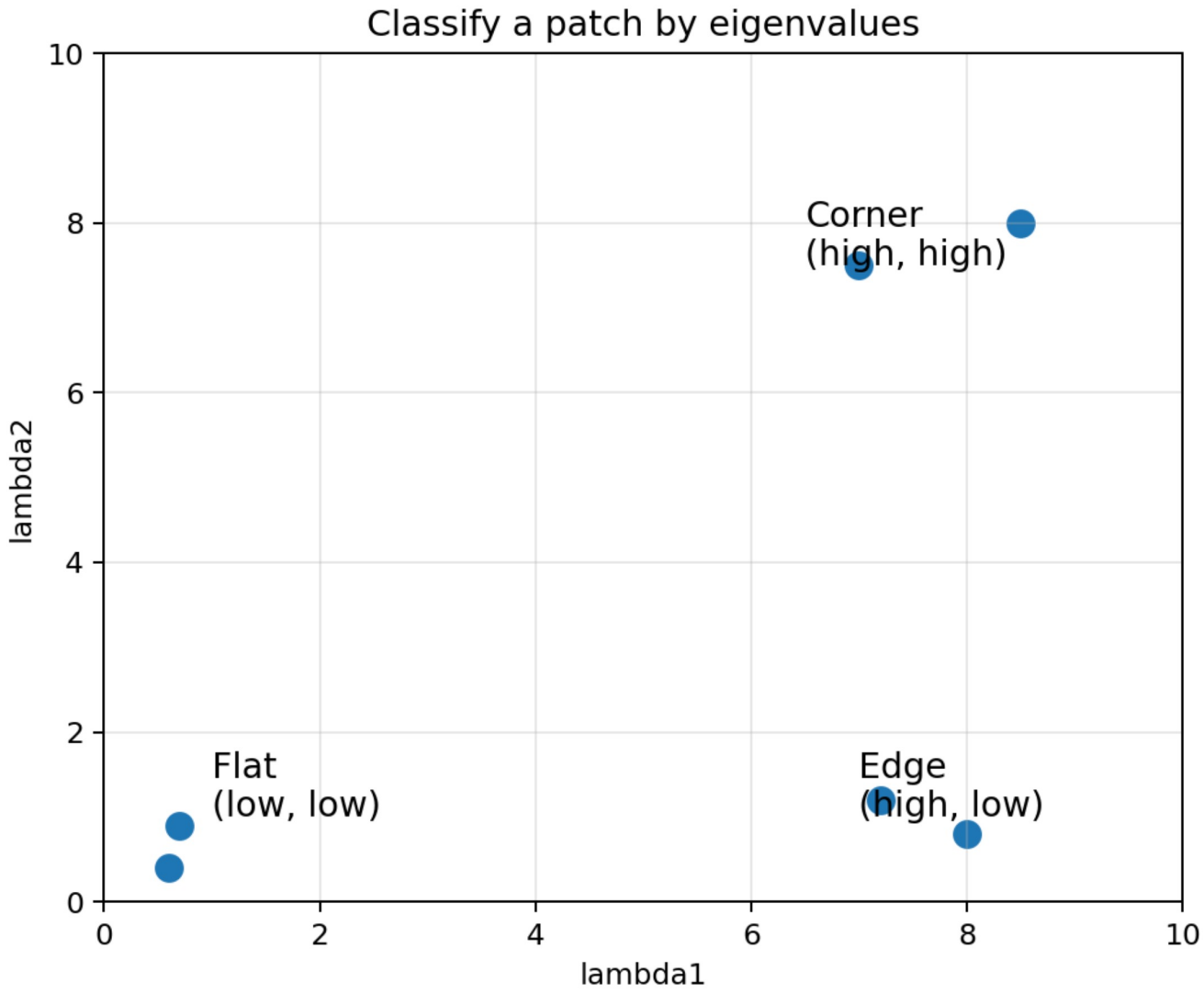
$$R = \det(M) - k \cdot \mathrm{trace}(M)^2 = \lambda_1 \lambda_2 - k(\lambda_1 + \lambda_2)^2$$

No eigenvalue solving needed — compute directly from matrix elements!



Eigenvalue Plane: Visual Classification

- **Flat**: both λ_1, λ_2 small
- **Edge**: λ_1 large, $\lambda_2 \approx 0$
- **Corner**: both λ_1, λ_2 large





How R Classifies: Flat / Edge / Corner

$$R = \lambda_1 \lambda_2 - k(\lambda_1 + \lambda_2)^2$$

- **Corner** (both λ large): $R \gg 0$
- **Flat** (both small): $R \approx 0$
- **Edge** ($\lambda_2 \approx 0$): $R \approx -k\lambda_1^2 < 0$

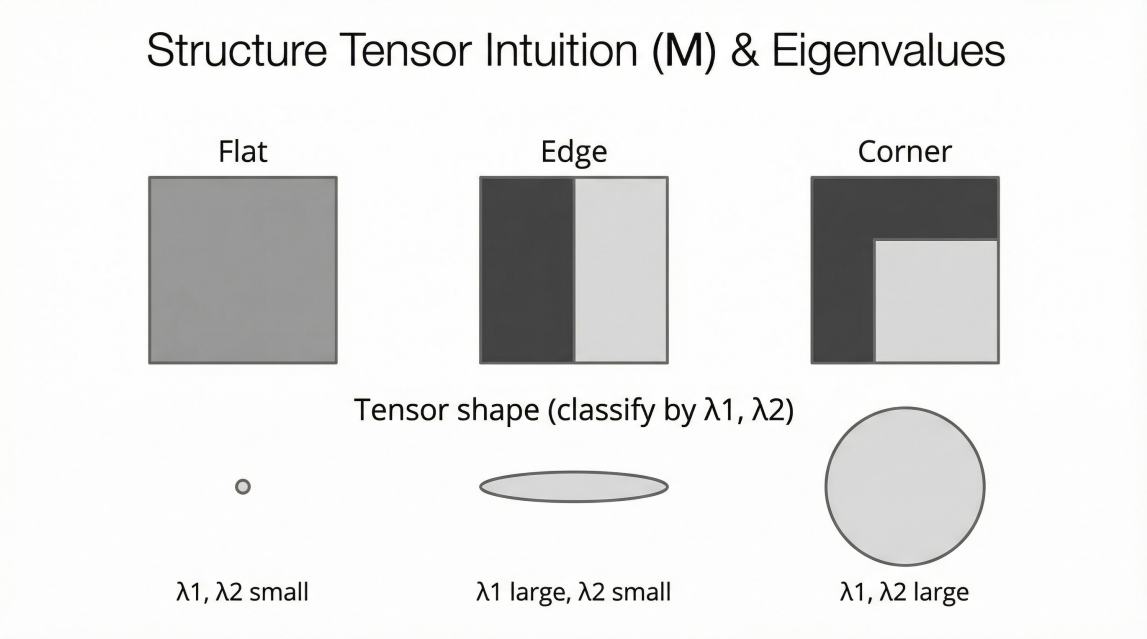
Typical k value: $0.04 \sim 0.06$



Structure Tensor Intuition

$$M = \sum_W \begin{bmatrix} I_x^2 & I_x I_y \\ I_x I_y & I_y^2 \end{bmatrix}$$

- **Flat**: $I_x \approx I_y \approx 0 \rightarrow M \approx 0$
- **Edge**: large gradient in one direction only
- **Corner**: large gradients in both directions



SIFT

Scale-Invariant Feature Transform

Harris detects keypoints, SIFT describes how the keypoint looks.



Problem 1: Scale Invariance

Question: the same corner looks different at different distances — how do we find keypoints that are stable across scales?

→ Answer: **Difference of Gaussians (DoG) scale space**

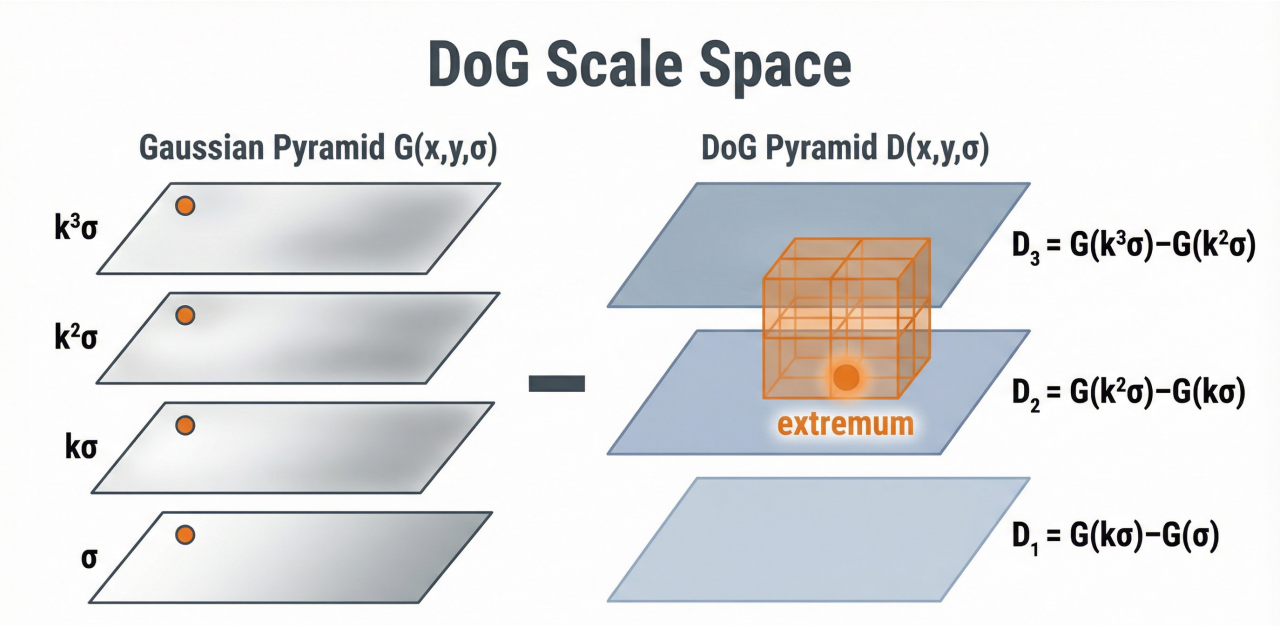


Scale Space: DoG

Build a **Gaussian pyramid** — blur at increasing σ, then subtract adjacent layers:

$$D(x, y, \sigma) = G(x, y, k\sigma) - G(x, y, \sigma)$$

- Find **3-D extrema** (x, y, scale) in the DoG volume — must be extreme among all 26 neighbours
- Each keypoint records its **σ** → descriptor patch size $\propto$ σ
- Same corner at different distances → matched at its "natural" scale



Problem 2: Rotation Invariance

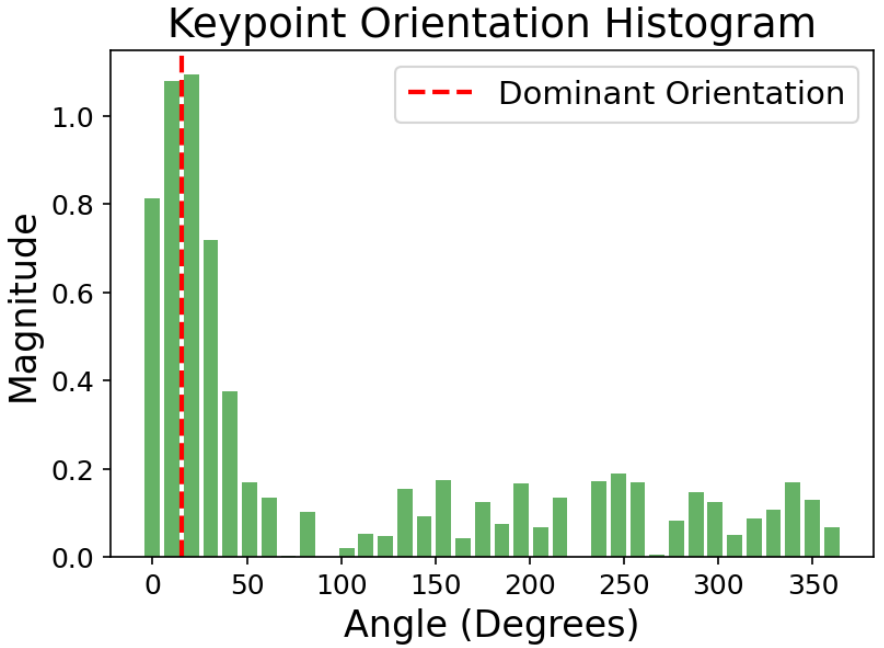
Question: if the camera rotates, the patch around a keypoint rotates too — how do we make the descriptor independent of orientation?

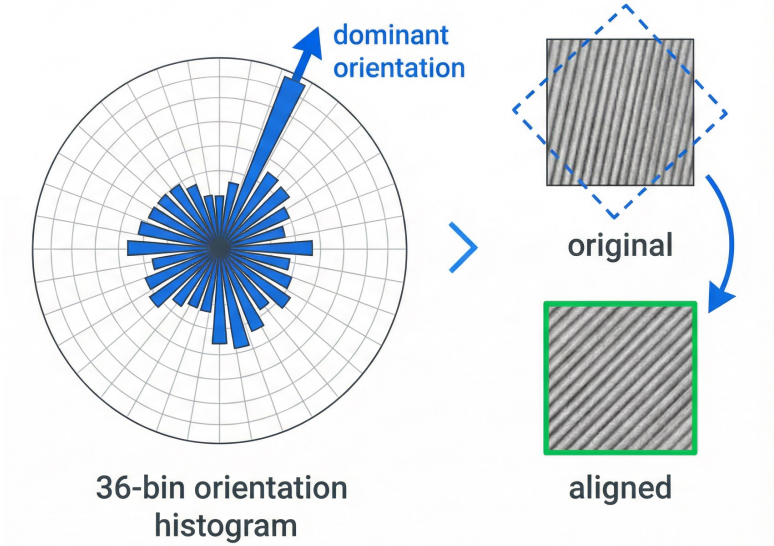
→ Answer: **align to the dominant gradient orientation before describing**



SIFT: Rotation Invariance

- **Dominant Orientation**:
  - Build a histogram of all gradients.
  - Highest peak is the **Reference Orientation**.
- **Normalization**:
  - Rotate the coordinate system to match this peak.
  - **Result**: "Up" is always relative to the feature's own structure.





Problem 3: Lighting Invariance

Question: now that we have a scale-normalised, orientation-aligned patch — what do we put in the descriptor?

Raw pixels fail under lighting change. We need a representation that is stable to brightness shifts.

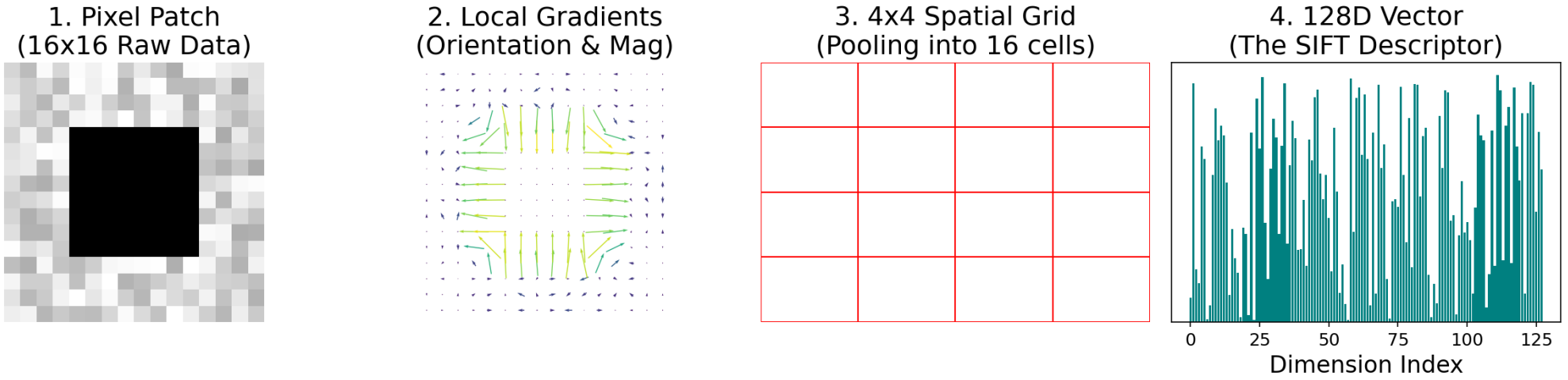
→ Answer: **gradient orientation histograms**



128-D Descriptor

Orientation-aligned, scale-normalised **16×16 neighbourhood**:

1. Divide into **4×4 = 16 cells** (each 4×4 px)
2. Per cell: gradient orientation histogram, **8 bins** (45° each)
3. Concatenate → 16 × 8 = **128-D vector**
4. L2-normalise → clip at 0.2 → re-normalise → lighting robust



Feature Matching

Brute Force → Ratio Test → RANSAC



Brute-Force Matching



For every descriptor, find the nearest neighbour in the other image → many false matches

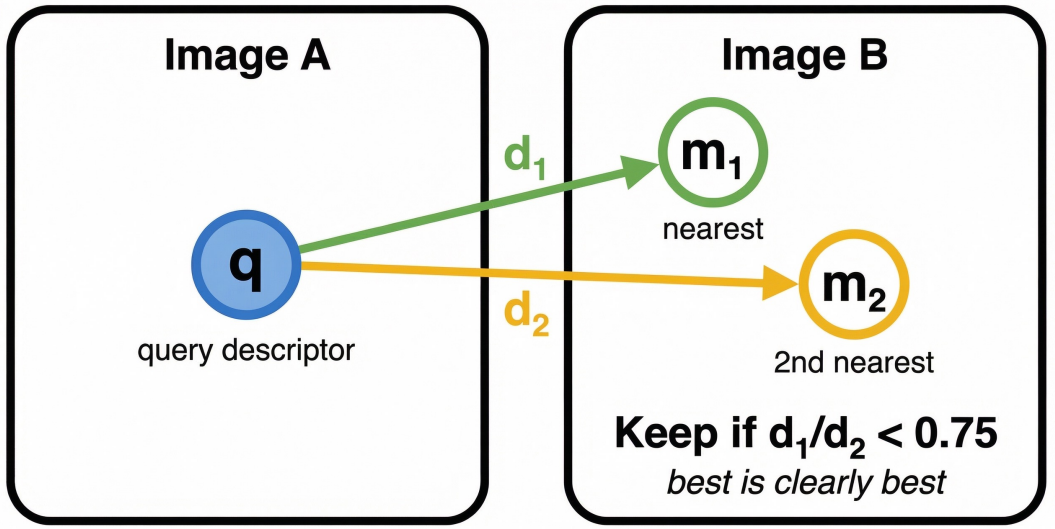


KNN + Lowe Ratio Test

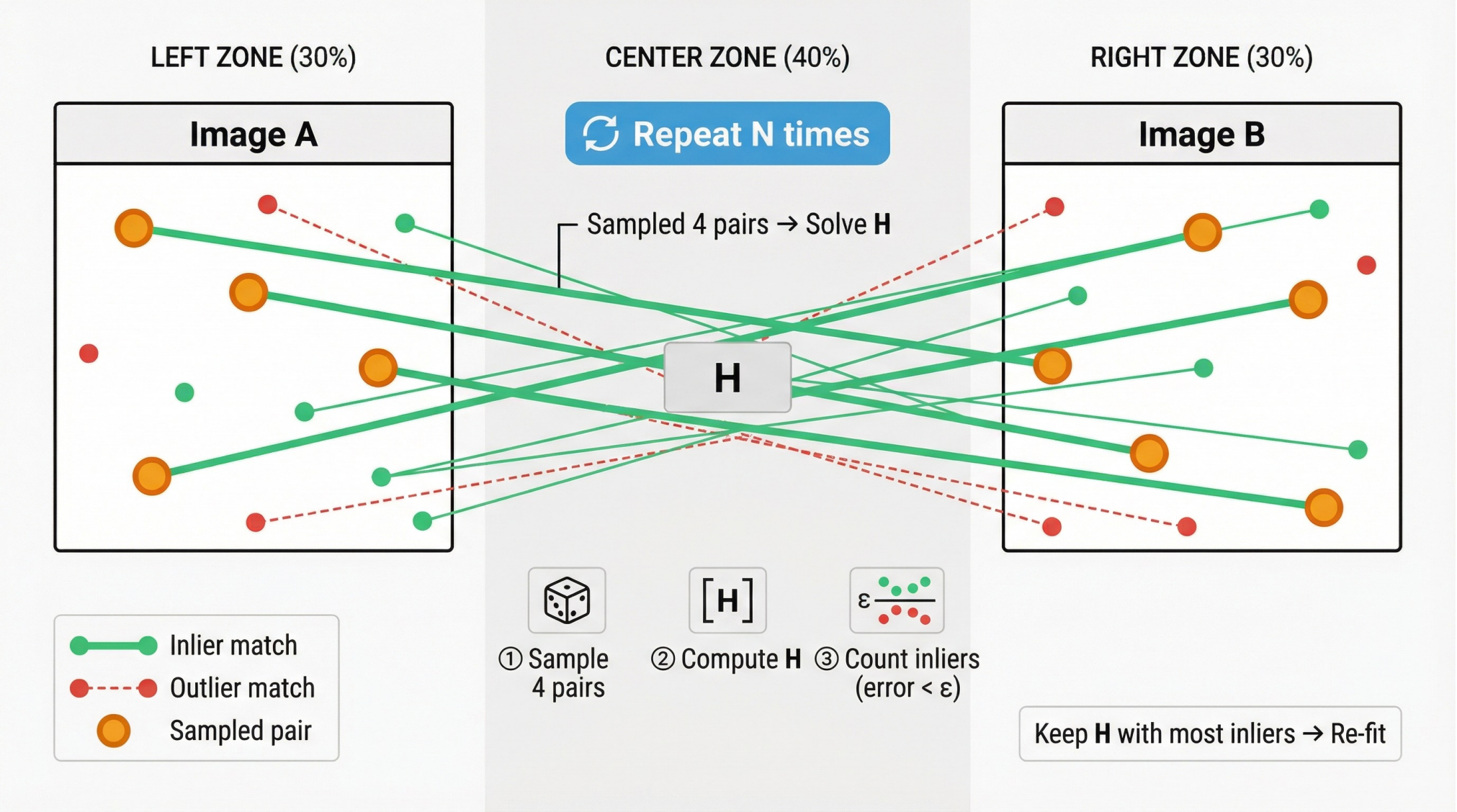
Find **k=2 nearest neighbours**, use the distance **ratio** as a confidence score:

$$\frac{d_1}{d_2} < 0.75 \quad \Rightarrow \quad \text{keep}$$

- **Good match**: $d_1 \ll d_2$, ratio near 0 → keep
- **Repetitive texture**: $d_1 \approx d_2$, ratio near 1 → discard
- **No correspondence**: both large and similar → discard



RANSAC Algorithm — Visual Overview



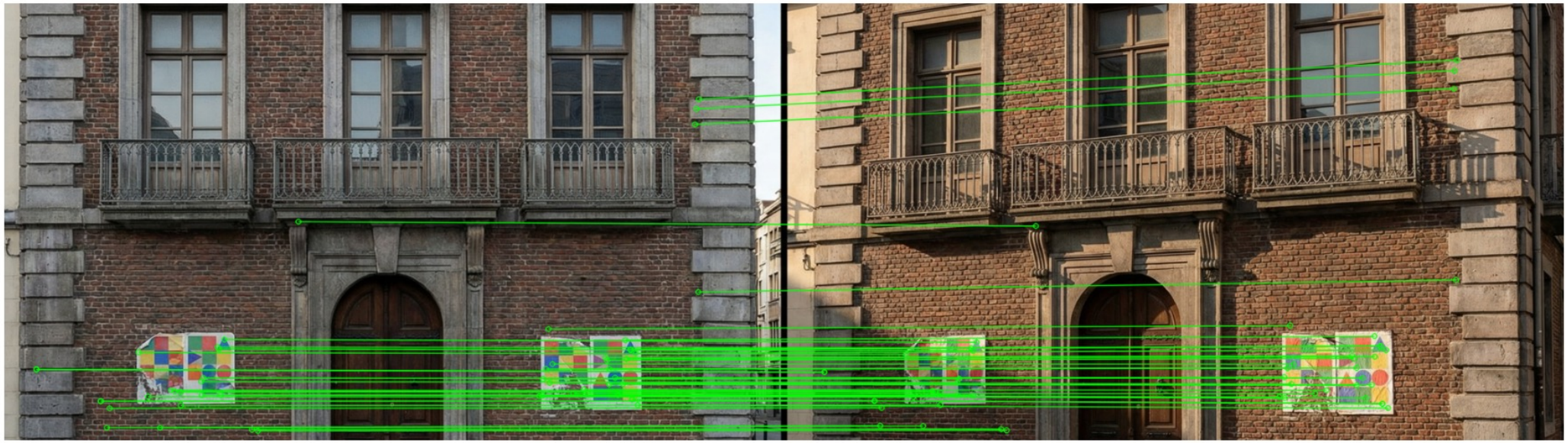
Homography Matrix

Projective transform between two views of a planar scene:

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} \sim H \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

- 8 degrees of freedom (9 elements − scale ambiguity)

Final: SIFT + Ratio + RANSAC



Today's Outline

- Linear Algebra
- K Nearest Neighbor
- Support Vector Machine
- Gradient descent
- Computation graph



Scalars

- **Simple operations**

$$c = a + b$$

$$c = a \cdot b$$

$$c = \sin a$$

- **Length**

$$|a| = \begin{cases} a & \text{if} a > 0 \\ -a & \text{otherwise} \end{cases}$$

$$|a + b| \leq |a| + |b|$$

$$|a \cdot b| = |a| \cdot |b|$$



Vectors

- **Simple operations**

Element-wise operation

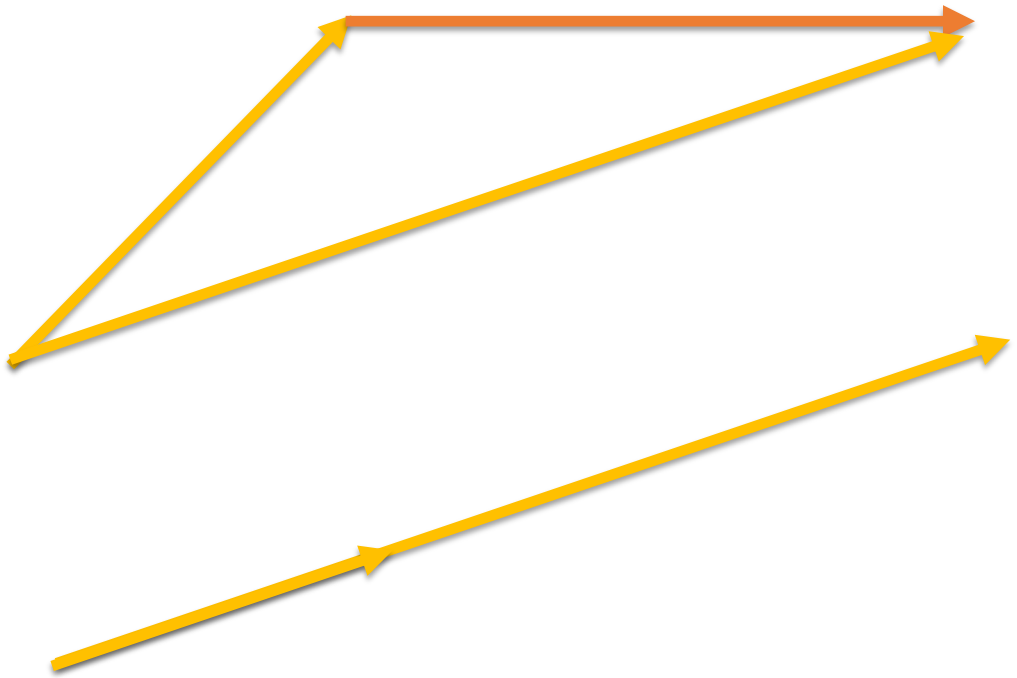
$$\boldsymbol{c} = \boldsymbol{a} + \boldsymbol{b} \text{ where } c_i = a_i + b_i$$
$$\boldsymbol{c} = \boldsymbol{a} \cdot \boldsymbol{b} \text{ where } c_i = a_i b_i$$
$$\boldsymbol{c} = \sin \boldsymbol{a} \text{ where } c_i = \sin(a_i)$$

- **Length (Norm)**

$$||\boldsymbol{x}|| = \sqrt{\sum_i x_i^2}$$
$$||\boldsymbol{a} + \boldsymbol{b}|| \leq ||\boldsymbol{a}|| + ||\boldsymbol{b}||$$
$$||a \cdot \boldsymbol{b}|| = |a| \cdot ||\boldsymbol{b}||$$



Vectors



$$c = a + b$$

$$c = \alpha \cdot b$$

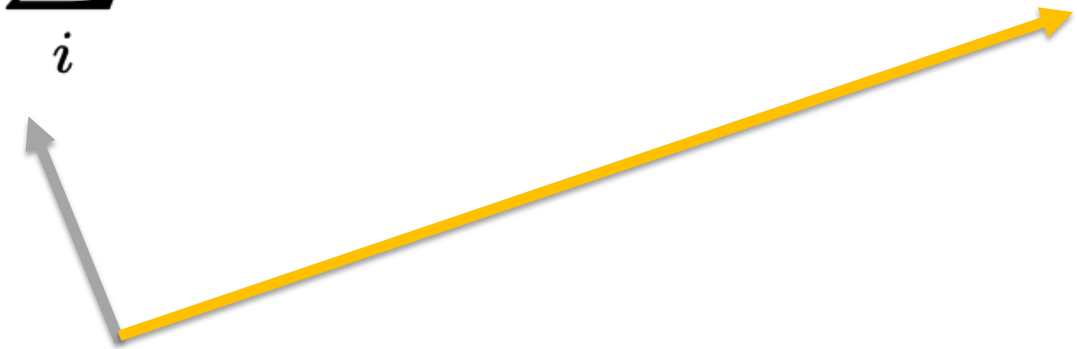
Vectors

- **Dot product**

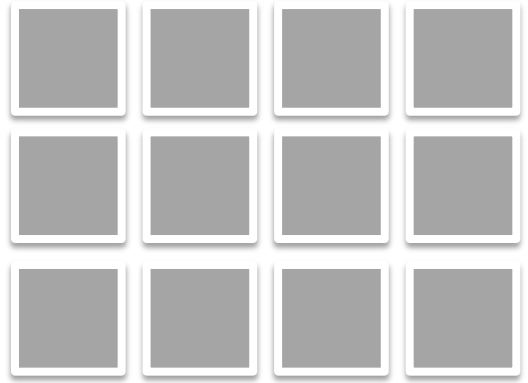
$$a^\top b = \sum_i a_i b_i$$

- **Orthogonal: dot product = 0**

$$a^\top b = \sum_i a_i b_i = 0$$



Matrices



- **Simple operations**

$$C = A + B \qquad \text{where } C_{ij} = A_{ij} + B_{ij}$$

$$C = \alpha \cdot B \qquad \text{where } C_{ij} = \alpha B_{ij}$$

$$C = \sin A \qquad \text{where } C_{ij} = \sin A_{ij}$$



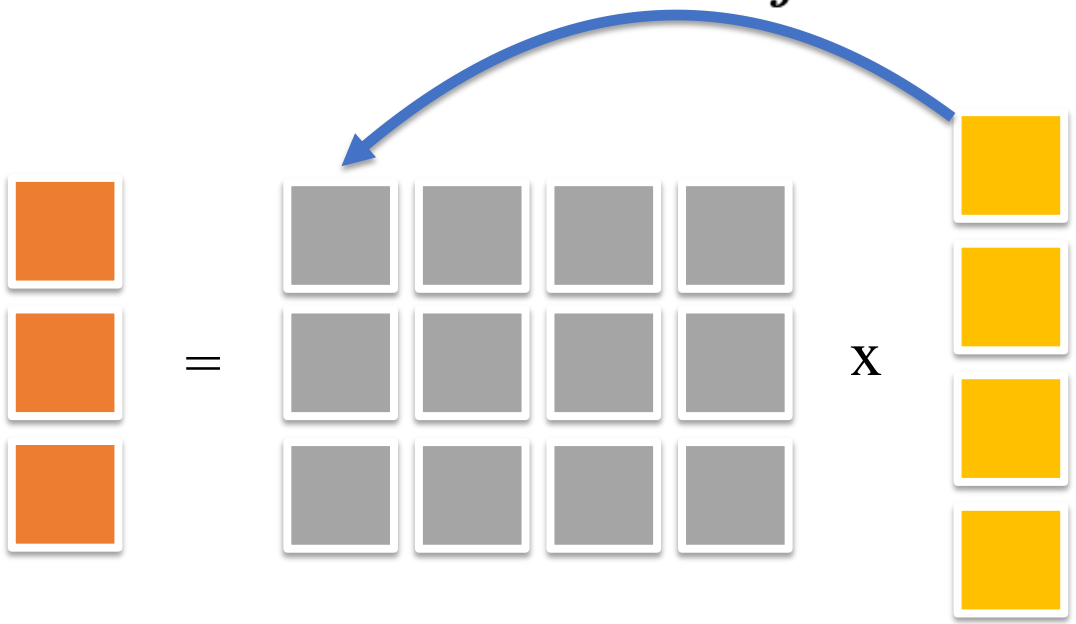
Matrices

- **Multiplications (matrix vector)**

$$c = Ab \text{ where } c_i = \sum_j A_{ij} b_j$$

Shape: Axb matmul b x c→ A x C

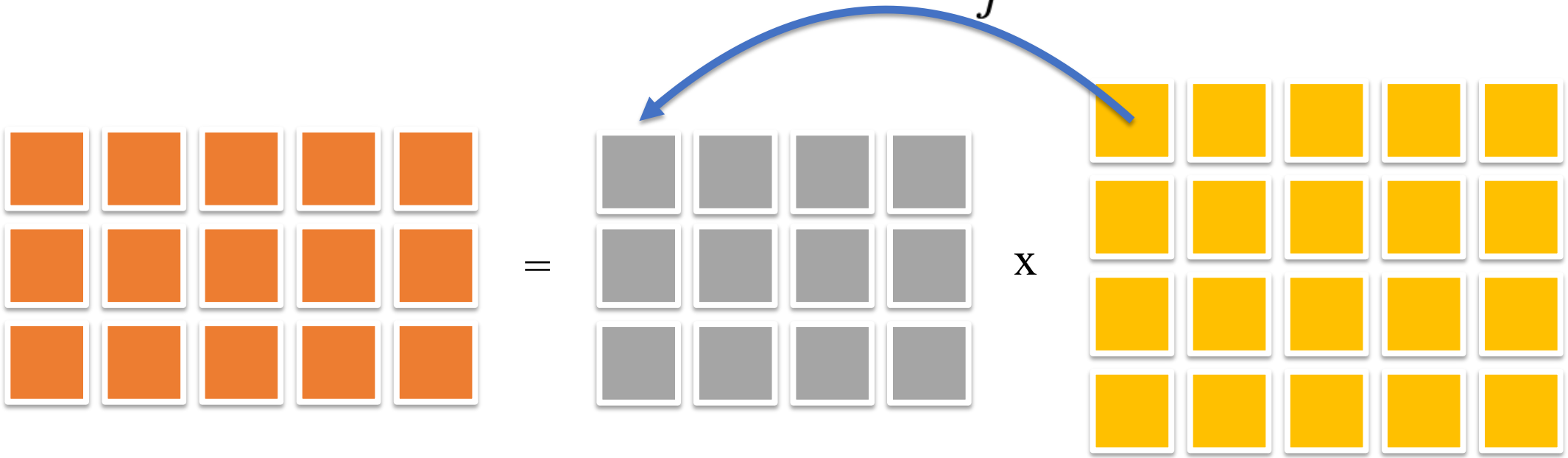
Axbxc matmul cxd -> Axbxd



Matrices

- **Multiplications (matrix matrix)**

$$C = AB \text{ where } C_{ik} = \sum_j A_{ij} B_{jk}$$



Matrices

- **Length (Norm)**

$$||A||_{Frob} = \left[\sum_{ij} A_{ij}^2\right]^{\frac{1}{2}}$$

$$||A + B||_{Frob} \leq ||A||_{Frob} + ||B||_{Frob}$$

$$||aA||_{Frob} = |a| \cdot ||A||_{Frob}$$

Other norms: p-norm, nuclear norm, spectral norm…
https://en.wikipedia.org/wiki/Matrix_norm



Image Classification

K Nearest Neighbor

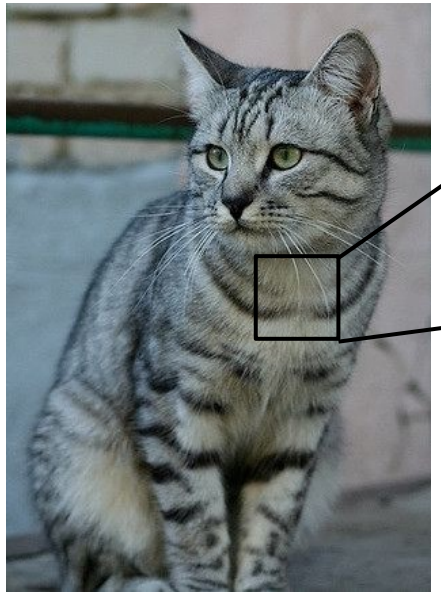


Problem



Cat

Problem



```
[[105 112 108 111 104  99 106  99  96 103 112 119 104  97  93  87]
 [ 91  98 102 106 104  79  98 103  99 105 123 136 110 105  94  85]
 [ 76  85  90 105 128 105  87  96  95  99 115 112 106 103  99  85]
 [ 99  81  81  93 120 131 127 100  95  98 102  99  96  93 101  94]
 [106  91  61  64  69  91  88  85 101 107 109  98  75  84  96  95]
 [114 108  85  55  55  69  64  54  64  87 112 129  98  74  84  91]
 [133 137 147 103  65  81  80  65  52  54  74  84 102  93  85  82]
 [128 137 144 140 109  95  86  70  62  65  63  63  60  73  86 101]
 [125 133 148 137 119 121 117  94  65  79  80  65  54  64  72  98]
 [127 125 131 147 133 127 126 131 111  96  89  75  61  64  72  84]
 [115 114 109 123 150 148 131 118 113 109 100  92  74  65  72  78]
 [ 89  93  90  97 108 147 131 118 113 114 113 109 106  95  77  80]
 [ 63  77  86  81  77  79 102 123 117 115 117 125 125 130 115  87]
 [ 62  65  82  89  78  71  80 101 124 126 119 101 107 114 131 119]
 [ 63  65  75  88  89  71  62  81 120 138 135 105  81  98 110 118]
 [ 87  65  71  87 106  95  69  45  76 130 126 107  92  94 105 112]
 [118  97  82  86 117 123 116  66  41  51  95  93  89  95 102 107]
 [164 146 112  80  82 120 124 104  76  48  45  66  88 101 102 109]
 [157 170 157 120  93  86 114 132 112  97  69  55  70  82  99  94]
 [130 128 134 161 139 100 109 118 121 134 114  87  65  53  69  86]
 [128 112  96 117 150 144 120 115 104 107 102  93  87  81  72  79]
 [123 107  96  86  83 112 153 149 122 109 104  75  80 107 112  99]
 [122 121 102  80  82  86  94 117 145 148 153 102  58  78  92 107]
 [122 164 148 103  71  56  78  83  93 103 119 139 102  61  69  84]]
```

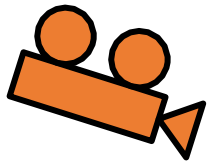
What the computer sees

An image is a tensor of integers between [0, 255]:

e.g. 800 x 600 x 3
(3 channels RGB)

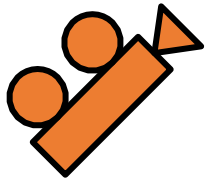


Challenges





```
[[105 112 100 111 104  99 106  99  96 103 112 119 104  97  93  87]
 [ 91  98 102 106 104  79  98 103  99 105 123 136 110 105  94  85]
 [ 76  85  90 105 128 105  87  96  95  99 115 112 106 103  99  85]
 [ 99  81  81  93 120 131 127 100  95  98 102  99  96  93 101  94]
 [106  91  61  64  69  91  88  85 101 107 109  98  75  84  96  95]
 [114 108  85  55  55  69  64  54  64  87 112 129  98  74  84  91]
 [133 137 147 103  65  81  80  65  52  54  74  84 102  93  85  82]
 [128 137 144 140 109  95  86  70  62  65  63  63  60  73  86 101]
 [125 133 148 137 119 121 117  94  65  79  80  65  54  64  72  98]
 [127 125 131 147 133 127 126 131 111  96  89  75  61  64  72  84]
 [115 114 109 123 150 148 131 118 113 109 100  92  74  65  72  70]
 [ 89  93  90  97 108 147 131 118 113 114 113 109 106  95  77  80]
 [ 63  77  86  81  77  79 102 123 117 115 117 125 125 130 115  87]
 [ 62  65  82  89  78  71  80 101 124 126 119 101 107 114 131 119]
 [ 63  65  75  88  89  71  62  81 120 138 135 105  81  98 110 118]
 [ 87  65  71  87 106  95  69  45  76 130 126 107  92  94 105 112]
 [118  97  82  86 117 123 116  66  41  51  95  93  89  95 102 107]
 [164 146 112  80  82 120 124 104  76  48  45  66  88 101 102 109]
 [157 170 157 120  93  86 114 132 112  97  69  55  70  82  99  94]
 [130 128 134 161 139 100 109 118 121 134 114  87  65  53  69  86]
 [128 112  96 117 150 144 120 115 104 107 102  93  87  81  72  79]
 [123 107  96  86  83 112 153 149 122 109 104  75  80 107 112  99]
 [122 121 102  80  82  86  94 117 145 148 153 102  58  78  92 107]
 [122 164 148 103  71  56  78  83  93 103 119 139 102  61  69  84]]
```





All pixels change when the camera moves!

Challenges: Background Clutter







Challenges: illumination



Challenges: Occlusion



Challenges: Deformation





Challenges: Intraclass variation



How do we classify an image?

```
def classify_image(image):
  # Some magic here?
  return class_label
```

Unlike e.g. sorting a list of numbers,

no obvious way to hard-code the algorithm for recognizing a cat, or other classes.

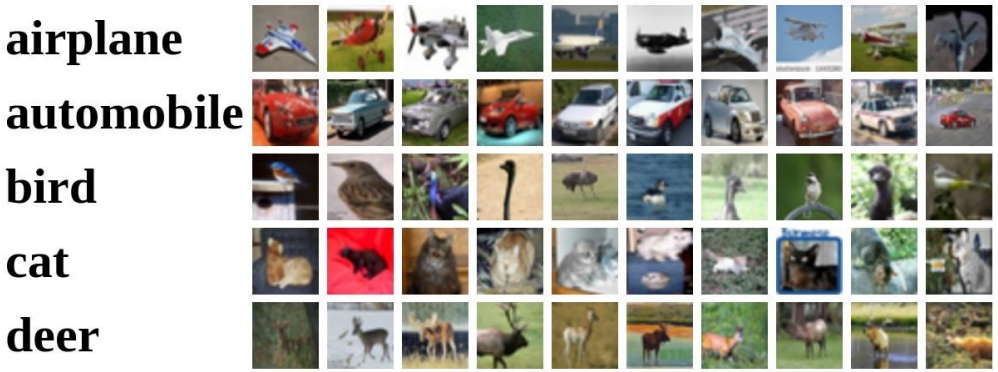


How do we classify an image?

1. Collect a dataset of images and labels
2. Extract discriminative features of the images
3. Use machine learning algorithms to train a classifier
4. Evaluate the classifier on new images

```
def train(images, labels):
  # Machine learning!
  return model
```

```
def predict(model, test_images):
  # Use model to predict labels
  return test_labels
```





Nearest neighbor

```
def train(images, labels):
  # Machine learning!
  return model
```



Memorize all data and labels

```
def predict(model, test_images):
  # Use model to predict labels
  return test_labels
```



Predict the label of the most similar training image



Nearest neighbor

deer bird plane cat car



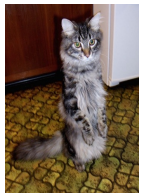
Training data with labels

?



query image

Distance Metric $|\ \ ,\ \ | \rightarrow R$





Distance Metric to compare images

L1 distance: $$d_1(I_1, I_2) = \sum_p |I_1^p - I_2^p|$$

test image					training image					pixel-wise absolute value differences					
56	32	10	18	-	10	20	24	17	=	46	12	14	1	add →	456
90	23	128	133		8	10	89	100		82	13	39	33		
24	26	178	200		12	16	178	170		12	10	0	30		
2	0	255	220		4	32	233	112		2	32	22	108		



```python
import numpy as np

class NearestNeighbor:
  def __init__(self):
    pass

  def train(self, X, y):
    """ X is N x D where each row is an example. Y is 1-dimension of size N """
    # the nearest neighbor classifier simply remembers all the training data
    self.Xtr = X
    self.ytr = y

  def predict(self, X):
    """ X is N x D where each row is an example we wish to predict label for """
    num_test = X.shape[0]
    # lets make sure that the output type matches the input type
    Ypred = np.zeros(num_test, dtype = self.ytr.dtype)

    # loop over all test rows
    for i in xrange(num_test):
      # find the nearest training image to the i'th test image
      # using the L1 distance (sum of absolute value differences)
      distances = np.sum(np.abs(self.Xtr - X[i,:]), axis = 1)
      min_index = np.argmin(distances) # get the index with smallest distance
      Ypred[i] = self.ytr[min_index] # predict the label of the nearest example

    return Ypred
```

Nearest Neighbor classifier

```python
import numpy as np

class NearestNeighbor:
  def __init__(self):
    pass

  def train(self, X, y):
    """ X is N x D where each row is an example. Y is 1-dimension of size N """
    # the nearest neighbor classifier simply remembers all the training data
    self.Xtr = X
    self.ytr = y

  def predict(self, X):
    """ X is N x D where each row is an example we wish to predict label for """
    num_test = X.shape[0]
    # lets make sure that the output type matches the input type
    Ypred = np.zeros(num_test, dtype = self.ytr.dtype)

    # loop over all test rows
    for i in xrange(num_test):
      # find the nearest training image to the i'th test image
      # using the L1 distance (sum of absolute value differences)
      distances = np.sum(np.abs(self.Xtr - X[i,:]), axis = 1)
      min_index = np.argmin(distances) # get the index with smallest distance
      Ypred[i] = self.ytr[min_index] # predict the label of the nearest example

    return Ypred
```

Nearest Neighbor classifier

Memorize training data

```python
import numpy as np

class NearestNeighbor:
  def __init__(self):
    pass

  def train(self, X, y):
    """ X is N x D where each row is an example. Y is 1-dimension of size N """
    # the nearest neighbor classifier simply remembers all the training data
    self.Xtr = X
    self.ytr = y

  def predict(self, X):
    """ X is N x D where each row is an example we wish to predict label for """
    num_test = X.shape[0]
    # lets make sure that the output type matches the input type
    Ypred = np.zeros(num_test, dtype = self.ytr.dtype)

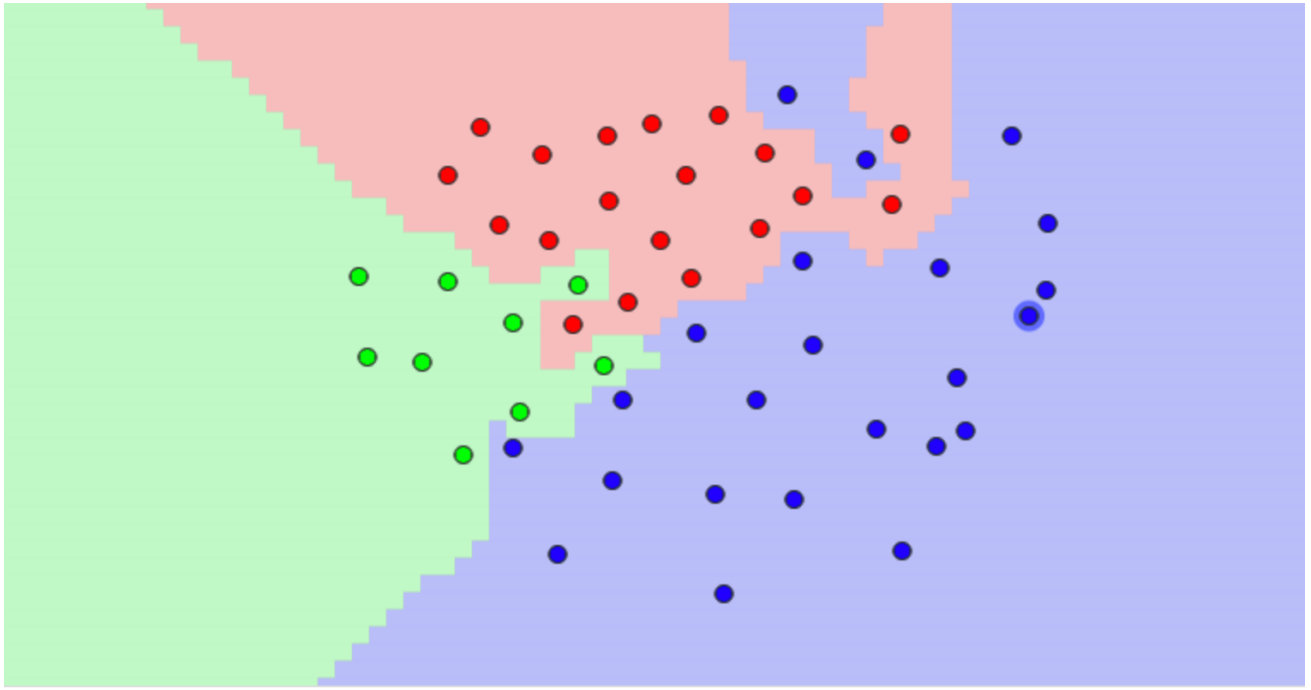
    # loop over all test rows
    for i in xrange(num_test):
      # find the nearest training image to the i'th test image
      # using the L1 distance (sum of absolute value differences)
      distances = np.sum(np.abs(self.Xtr - X[i,:]), axis = 1)
      min_index = np.argmin(distances) # get the index with smallest distance
      Ypred[i] = self.ytr[min_index] # predict the label of the nearest example

    return Ypred
```

Nearest Neighbor classifier

For each test image:
Find closest train image
Predict label of nearest image

What does this looks like?

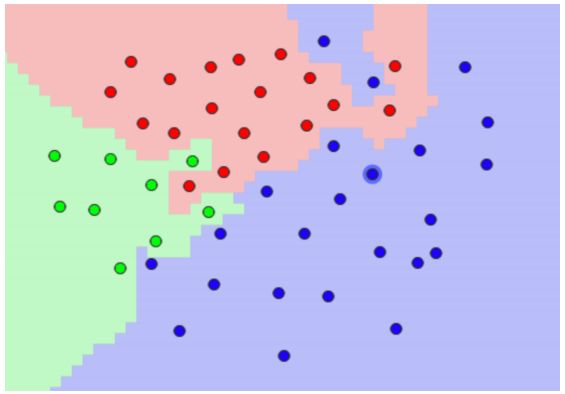


1-nearest neighbor

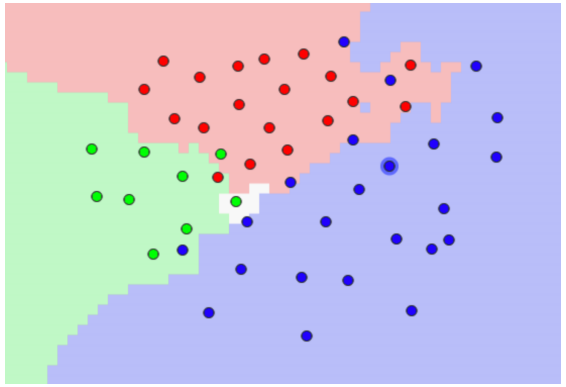


From Nearest neighbor to K Nearest Neighbor

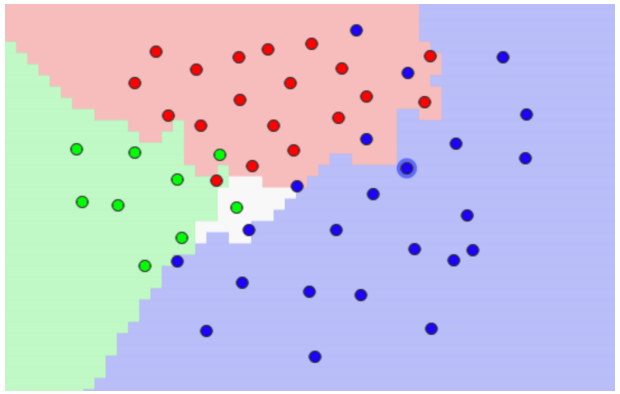
Instead of copying label from nearest neighbor,
take **majority vote** from K closest points



K = 1



K = 3



K = 5

Is a larger k always better?

Hyperparameters matters!

What is the best value of **k** to use? What is the best **distance** to use?

These are **hyperparameters**: choices about the algorithms themselves.

Very problem/dataset-dependent.
Must try them all out and see what works best.



Setting hyperparameters

Idea #1: Choose hyperparameters that work best on the **training data**

BAD: K = 1 always works perfectly on training data





Setting hyperparameters

Idea #1: Choose hyperparameters that work best on the **training data**

BAD: K = 1 always works perfectly on training data



Idea #2: choose hyperparameters that work best on **test** data

BAD: No idea how algorithm will perform on new data





Setting hyperparameters

Idea #1: Choose hyperparameters that work best on the **training data**

BAD: K = 1 always works perfectly on training data



Idea #2: choose hyperparameters that work best on **test** data

BAD: No idea how algorithm will perform on new data



Idea #3: Split data into **train**, **val**; choose hyperparameters on val and evaluate on test

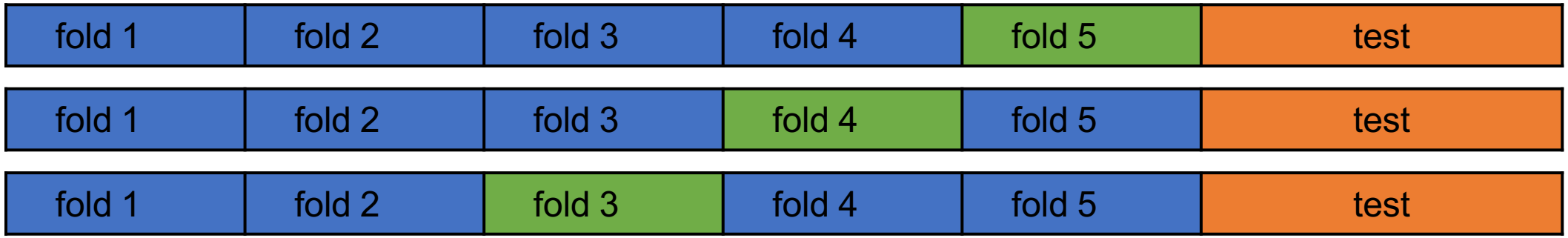
Better!





Setting hyperparameters

Idea #4: **Cross-Validation**: Split data into **folds**,
try each fold as validation and average the results



Useful for small datasets, but not used too frequently in deep learning



Case: A student reports 92% test accuracy

A student tries:

- k = 1, 3, 5, 7, …, 99
- 10 distance metrics
- 20 preprocessing methods

Is 92% trustworthy?

No! The test set has effectively become the validation set.

He chooses the setting with the highest test accuracy.

Final reported accuracy: 92%

You can only touch the test set once!



Results of KNN in **CIFAR10**

10 classes
50,000 training images
10,000 testing images



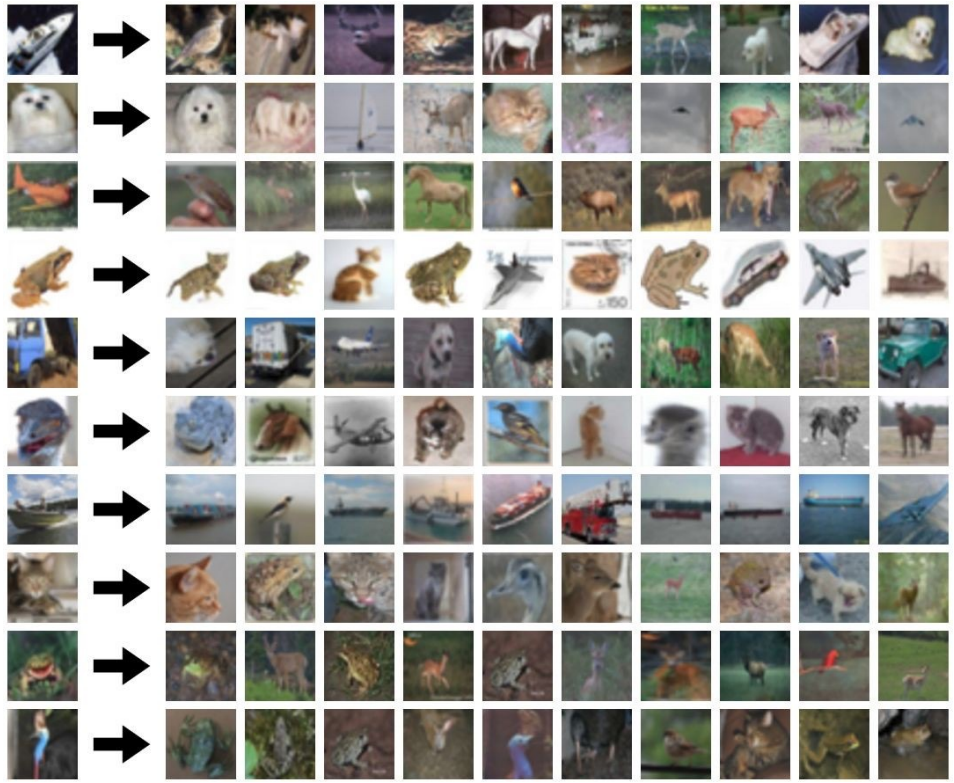


Results of KNN in **CIFAR10**

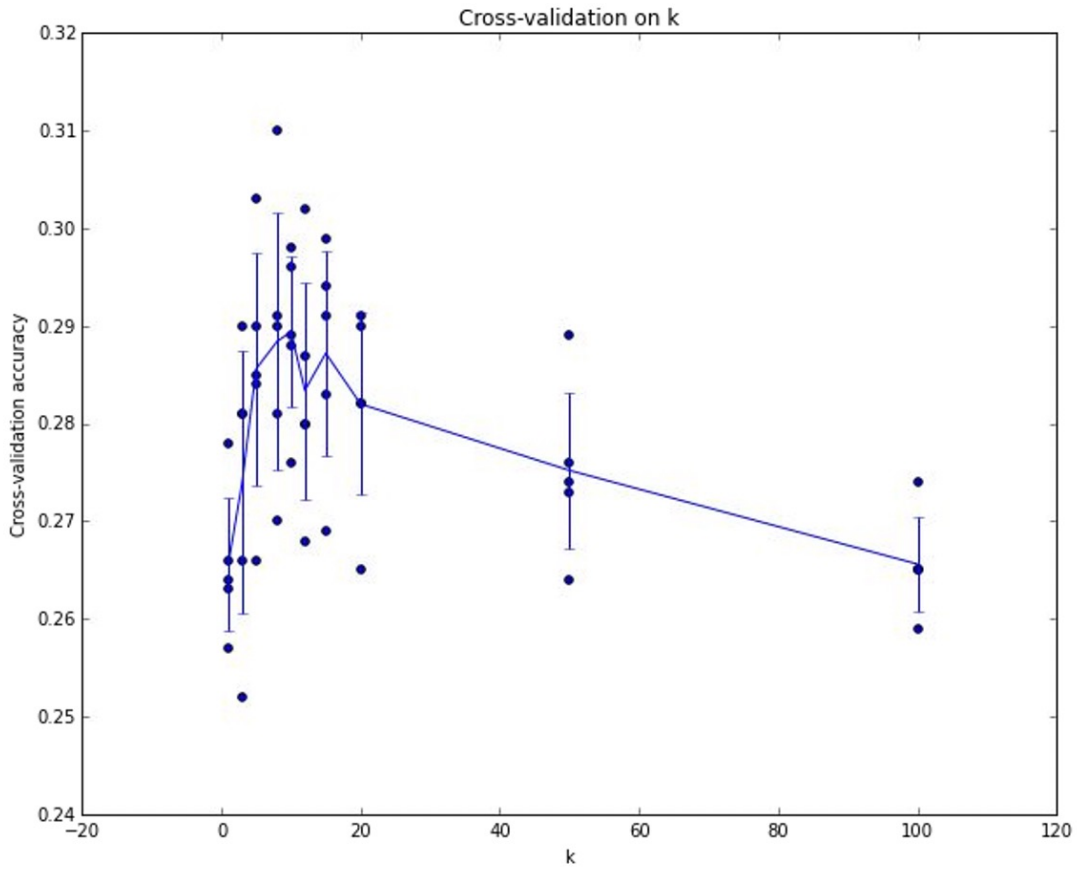
10 classes
50,000 training images
10,000 testing images



Test images with raw pixel feature and nearest neighbors



Setting hyperparameters



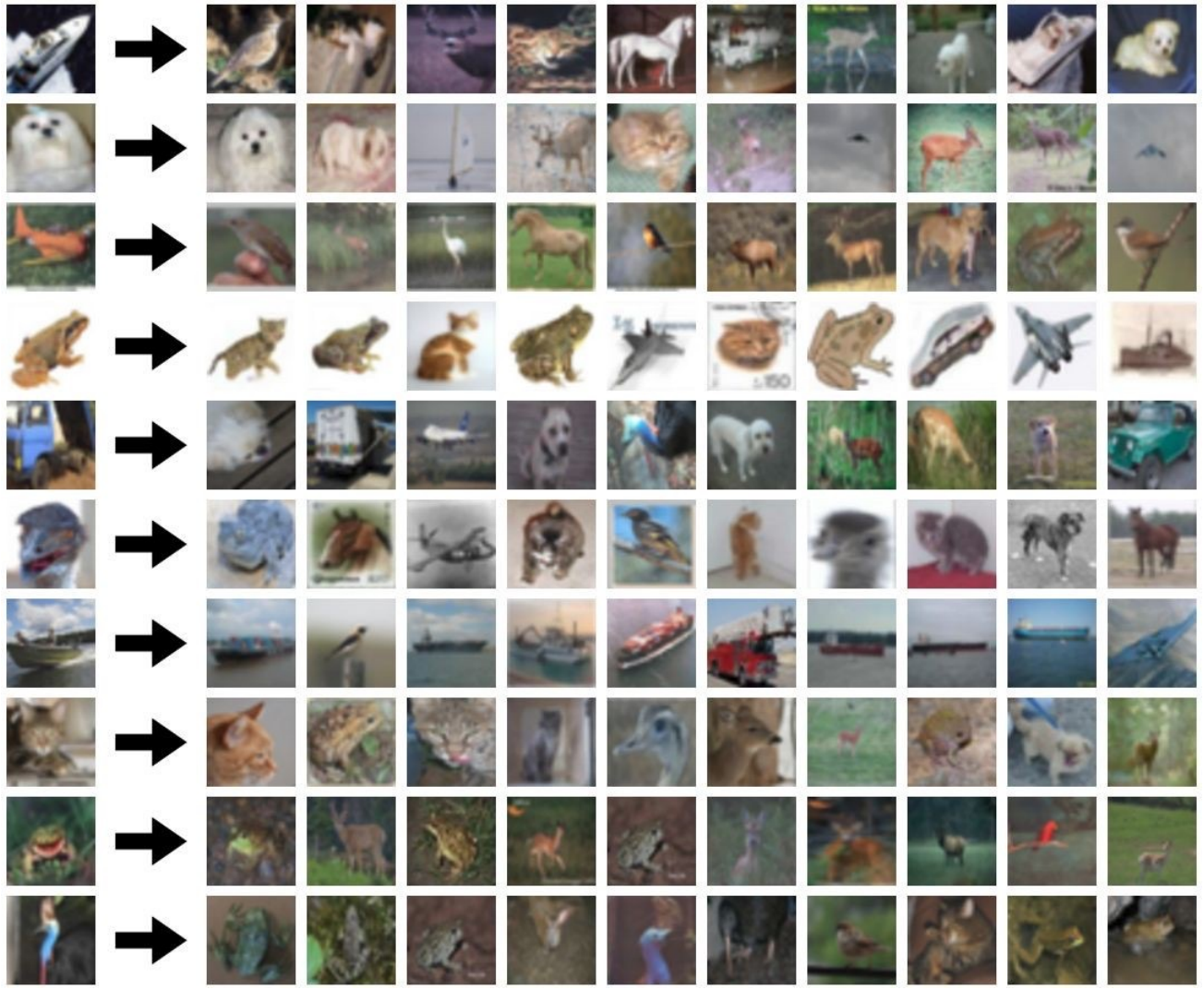
Example of
5-fold cross-validation for the value of **k.**

Each point: single outcome.

The line goes
through the mean, bars indicated standard deviation

(Seems that k ~= 7 works best for this data)

What does this look like?



What does this look like?



k-Nearest Neighbor with pixel distance **never used.**

- Distance metrics on pixels are not informative
- Very slow at test time

Feature extraction is needed!

Would you compare these images pixel by pixel?

Semantic similarity ≠ pixel similarity



Original



Occluded



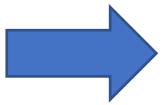
Shuffled (1 pixel)



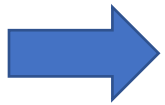
Recolored

How do we classify an image?





Feature Extraction



Machine learning



How do we classify an image?



Feature Extraction

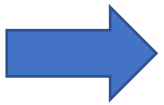
Machine learning

Raw pixels
Histogram of Oriented Gradient (HoG) [1]
Local Binary Pattern (LBP) [2]
Scale Invariant Feature Transform (SIFT) [3]
…

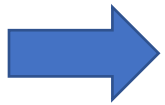
[1] “Histograms of oriented gradients for human detection.” CVPR 2015
[2] “A completed modeling of local binary pattern operator for texture classification”, TIP 2010
[3] “ ”SIFT: Predicting amino acid changes that affect protein function.“ Nucleic acids research, 2013

How do we classify an image?





Feature Extraction



Machine learning

Nearest Neighbor
Linear Classifier
Support Vector Machine
…

A Brief Summary

- In **image classification** we start with a **training set** of images and labels, and must predict labels on the **test set**
- The **K-Nearest Neighbors** classifier predicts labels based on the K nearest training examples
- Distance metric and K are **hyperparameters**

Choose hyperparameters using the **validation set**;

Only run on the test set once at the very end!



Linear Classifier



A more typical framework

K Nearest Neighbor

```
def train(images, labels):
  # Machine learning!
  return model
```



Memorize all data and labels

```
def predict(model, test_images):
  # Use model to predict labels
  return test_labels
```



Predict the label of the most similar training image



A more typical framework

Linear classifier

```
def train(images, labels):
  # Machine learning!
  return model
```



Define a model $f_W(x)$
Define a loss function $L(f_W(x), y)$
Solve W^* that leads to minimal L

```
def predict(model, test_images):
  # Use model to predict labels
  return test_labels
```



Predict the labels based on $f_{W^*}(x)$



A more typical framework

Linear classifier

```
def train(images, labels):
  # Machine learning!
  return model
```



Define a model $f_W(x)$

Define a loss function $L(f_W(x), y)$

Solve W^* that leads to minimal L

```
def predict(model, test_images):
  # Use model to predict labels
  return test_labels
```



Predict the labels based on $f_{W^*}(x)$

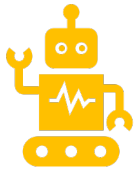


Two Ways to Learn



KNN:
"I remember everything."

- Store all training images
- Compare at test time
- Almost no "training"



Linear Classifier:
"I summarize what I learned."

- Learn a parameterized model W for features
- Discard training images at inference
- Predict with W

Which one is more like "learning"?

Problem formulation



$I \in R^{32\times32\times3}$



$\boldsymbol{x} \in R^{3072\times1}$

vectorization

$f(\boldsymbol{x}, \boldsymbol{W}, \boldsymbol{b})$

$\boldsymbol{W}, \boldsymbol{b}$

parameters

10 numbers giving class scores

Prediction: largest score



Problem formulation



$I \in R^{32\times32\times3}$



$\boldsymbol{x} \in R^{3073\times1}$

vectorization

$f(\boldsymbol{x}, \boldsymbol{W})$

parameters

10 numbers giving class scores

Prediction: largest score

By augmenting the feature, we can remove the bias.



A more typical framework

Linear classifier

```
def train(images, labels):
  # Machine learning!
  return model
```



Define a model $f_W(x)$
Define a loss function $L(f_W(x), y)$
Solve W^* that leads to minimal L

```
def predict(model, test_images):
  # Use model to predict labels
  return test_labels
```



Predict the labels based on $f_{W^*}(x)$



Solving $\boldsymbol{w}$ and b with mult-class SVM loss





cat	**3.2**	1.3	2.2
car	5.1	**4.9**	2.5
frog	-1.7	2.0	**-3.1**

Given a dataset of examples

$$\{x_i, y_i\}_{i=1}^{N}$$

Where x_i is the image and y_i is the label.

$$L = \frac{1}{N}\sum_i L_i(f(x_i, \boldsymbol{W}), y_i)$$

Multiclass SVM loss:

$$L_i = \sum_{j \neq y_i} \begin{cases} 0 & if\ s_{y_i} > s_j + 1 \\ s_j - s_{y_i} + 1 & othersise \end{cases}$$

where $\boldsymbol{s} = f(x_i, \boldsymbol{W}) \in R^3$ is the prediction score.

Solving $\boldsymbol{w}$ and b with mult-class SVM loss

Both predict "Cat". Are they equally good?





cat	**5.0**	**5.0**
car	4.9	1.5
frog	1.0	1.0

Correct classification is not enough.
We also want a **margin**.

Solving $\boldsymbol{w}$ and b with mult-class SVM loss





cat	**3.2**	1.3	2.2
car	5.1	**4.9**	2.5
frog	-1.7	2.0	**-3.1**

What does it mean?

Multiclass SVM loss:

$$L_i = \sum_{j \neq y_i} \begin{cases} 0 & if\ s_{y_i} > s_j + 1 \\ s_j - s_{y_i} + 1 & othersise \end{cases}$$

where $\boldsymbol{s} = f(x_i, \boldsymbol{W}) \in R^3$ is the prediction score.

Solving $\boldsymbol{w}$ and b with mult-class SVM loss





cat	**3.2**	1.3	2.2
car	5.1	**4.9**	2.5
frog	-1.7	2.0	**-3.1**

Multiclass SVM loss:

$$L_i = \sum_{j \neq y_i} \begin{cases} 0 & if\ s_{y_i} > s_j + 1 \\ s_j - s_{y_i} + 1 & othersise \end{cases}$$

For all scores corresponding to classes other than target one

Solving $\boldsymbol{w}$ and b with mult-class SVM loss





cat	**3.2**	1.3	2.2
car	5.1	**4.9**	2.5
frog	-1.7	2.0	**-3.1**

Multiclass SVM loss:

$$L_i = \sum_{j \neq y_i} \begin{cases} 0 & if\ s_{y_i} > s_j + 1 \\ s_j - s_{y_i} + 1 & othersise \end{cases}$$

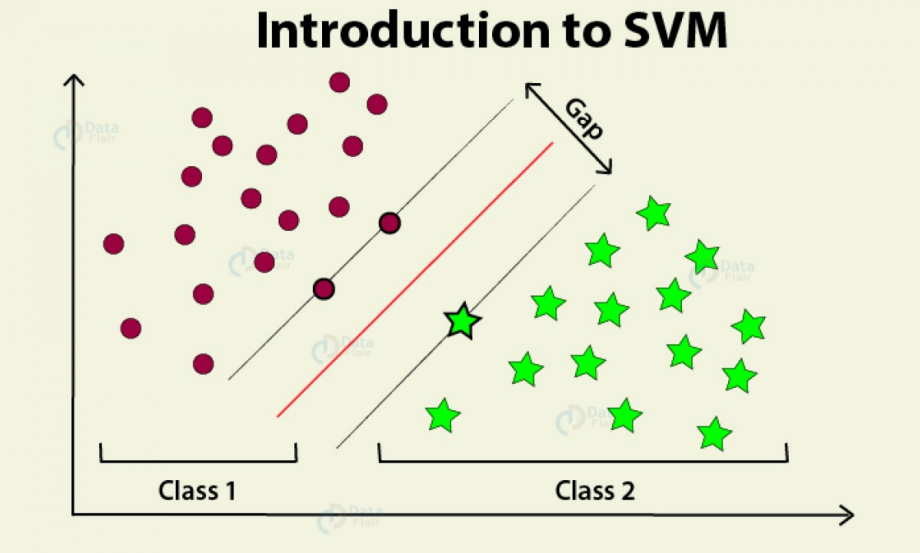
If the score value is smaller than that of target class plus a margin

3.2 > -1.7 + 1

Then the loss is 0

Solving $\boldsymbol{w}$ and b with mult-class SVM loss

Margin: we want to encourage each two classes be separated by a gap.



source: https://morioh.com/p/74f4a4b78e0b

Multiclass SVM loss:

$$L_i = \sum_{j \neq y_i} \begin{cases} 0 & if\ s_{y_i} > s_j + 1 \\ s_j - s_{y_i} + 1 & othersise \end{cases}$$

If the score value is smaller than that of target class plus a margin

$3.2 > -1.7 + 1$

Then the loss is 0



Solving $\boldsymbol{w}$ and b with mult-class SVM loss





cat	**3.2**	1.3	2.2
car	5.1	**4.9**	2.5
frog	-1.7	2.0	**-3.1**

Multiclass SVM loss:

$$L_i = \sum_{j \neq y_i} \begin{cases} 0 & if\ s_{y_i} > s_j + 1 \\ s_j - s_{y_i} + 1 & othersise \end{cases}$$

Otherwise

$3.2 < 5.1 + 1$

The loss is the difference between the current and target score plus a margin

Solving $\boldsymbol{w}$ and b with mult-class SVM loss





cat	**3.2**	1.3	2.2
car	5.1	**4.9**	2.5
frog	-1.7	2.0	**-3.1**

Multiclass SVM loss:

$$L_i = \sum_{j \neq y_i} \begin{cases} 0 & if\ s_{y_i} > s_j + 1 \\ s_j - s_{y_i} + 1 & othersise \end{cases}$$

Rewrite

$$L_i = \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + 1)$$

Solving $\boldsymbol{w}$ and b with mult-class SVM loss

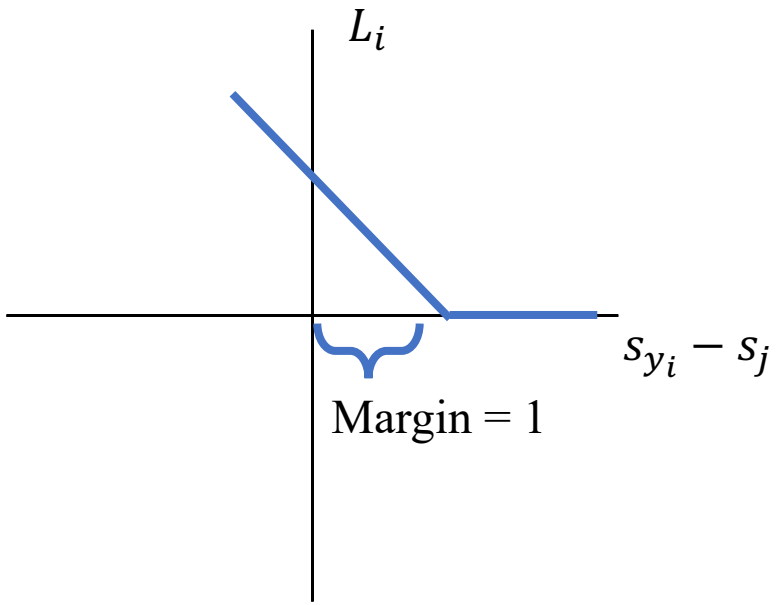




cat	**3.2**	1.3	2.2
car	5.1	**4.9**	2.5
frog	-1.7	2.0	**-3.1**

Multiclass SVM loss:

$$L_i = \sum_{j \neq y_i} \max\left(0, s_j - s_{y_i} + 1\right)$$



Solving $\boldsymbol{w}$ and b with mult-class SVM loss





cat	**3.2**	1.3	2.2
car	5.1	**4.9**	2.5
frog	-1.7	2.0	**-3.1**
Loss	2.9		

Multiclass SVM loss:

$$L_i = \sum_{j \neq y_i} \max\left(0, s_j - s_{y_i} + 1\right)$$

= max(0, 5.1 - 3.2 + 1)
+max(0, -1.7 - 3.2 + 1)
= max(0, 2.9) + max(0, -3.9)
= 2.9 + 0
= 2.9



Solving $\boldsymbol{w}$ and b with mult-class SVM loss





cat	**3.2**	1.3	2.2
car	5.1	**4.9**	2.5
frog	-1.7	2.0	**-3.1**
Loss	2.9	0	

Multiclass SVM loss:

$$L_i = \sum_{j \neq y_i} \max\left(0, s_j - s_{y_i} + 1\right)$$

= max(0, 1.3 - 4.9 + 1)
+max(0, 2.0 - 4.9 + 1)
= max(0, -2.6) + max(0, -1.9)
= 0 + 0
= 0



Solving $\boldsymbol{w}$ and b with mult-class SVM loss





cat	**3.2**	1.3	2.2
car	5.1	**4.9**	2.5
frog	-1.7	2.0	**-3.1**
Loss	2.9	0	12.9

Multiclass SVM loss:

$$L_i = \sum_{j \neq y_i} \max\left(0, s_j - s_{y_i} + 1\right)$$

= max(0, 2.2 - (-3.1) + 1)
+max(0, 2.5 - (-3.1) + 1)
= max(0, 6.3) + max(0, 6.6)
= 6.3 + 6.6
= 12.9



Solving $\boldsymbol{w}$ and b with mult-class SVM loss





cat	**3.2**	1.3	2.2
car	5.1	**4.9**	2.5
frog	-1.7	2.0	**-3.1**
Loss	2.9	0	12.9

Multiclass SVM loss:

$$L_i = \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + 1)$$

$$L = \frac{1}{N} \sum_{i=1}^{N} L_i$$

$$L = (2.9 + 0 + 12.9)/3$$
$$= \mathbf{5.27}$$

Example code

$$L_i = \sum_{j \neq y_i} \max\left(0, s_j - s_{y_i} + 1\right)$$

```
def L_i_vectorized(x, y, W):
  scores = W.dot(x)
  margins = np.maximum(0, scores - scores[y] + 1)
  margins[y] = 0
  loss_i = np.sum(margins)
  return loss_i
```



Is the optimal *W* unique?

$$f(\boldsymbol{x}; \boldsymbol{W}) = \boldsymbol{W}\boldsymbol{x}$$

$$L = \frac{1}{N}\sum_{i=1}^{N}\sum_{j \neq y_j} \max\left(0, f(\boldsymbol{x_i}; \boldsymbol{W})_j - f(\boldsymbol{x}_i; \boldsymbol{W})_{y_i} + 1\right)$$

Suppose we found a **W** such that L=0.
Is this **W** unique?

No! nW also lead to L=0.

How do we choose between W and nW?



Is the optimal W unique?

$$f(x; \boldsymbol{W}) = \boldsymbol{W}\boldsymbol{x}$$

$$L = \frac{1}{N}\sum_{i=1}^{N}\sum_{j \neq y_j} \max\left(0, [\boldsymbol{W}\boldsymbol{x}]_j - [\boldsymbol{W}\boldsymbol{x}]_{y_i} + 1\right)$$



Is the optimal W unique?

$$f(\boldsymbol{x}; \boldsymbol{W}) = \boldsymbol{W}\boldsymbol{x}$$

$$L = \frac{1}{N}\sum_{i=1}^{N}\sum_{j \neq y_j} \max\left(0, [n\boldsymbol{W}\boldsymbol{x}]_j - [n\boldsymbol{W}\boldsymbol{x}]_{y_i} + 1\right)$$



Is the optimal $\boldsymbol{W}$ unique?

$$f(\boldsymbol{x}; \boldsymbol{W}) = \boldsymbol{W}\boldsymbol{x}$$

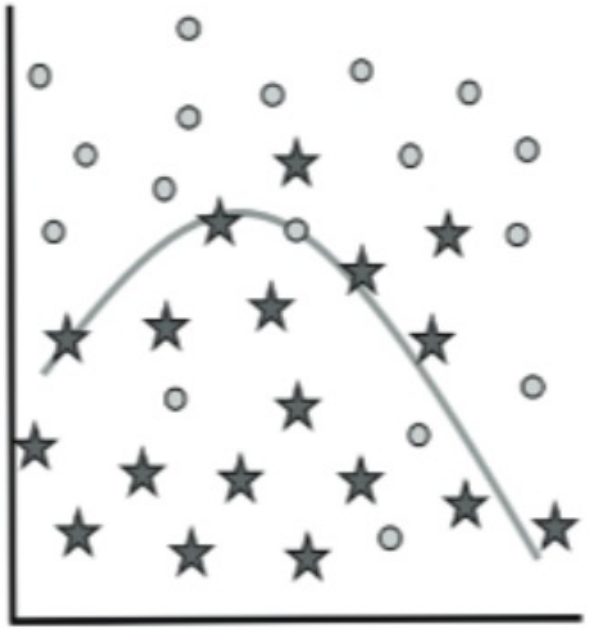
$$L = \frac{n}{N}\sum_{i=1}^{N}\sum_{j \neq y_j} \max\left(0, [\boldsymbol{W}\boldsymbol{x}]_j - [\boldsymbol{W}\boldsymbol{x}]_{y_i} + \frac{1}{n}\right)$$

As n increases, the margin gets small, which could lead to overfitting.

$$f(\boldsymbol{x}; \boldsymbol{W}) = \boldsymbol{W}\boldsymbol{x}$$



Which model would you deploy?





What if Model A has 96% training accuracy, Model B only has 100%?

Training accuracy is not the goal.
Generalization is.



Regularization

$$L = \frac{1}{N}\sum_{i=1}^{N} L_i(\boldsymbol{x_i}; \boldsymbol{W}), y_i)$$

Data loss: Model predictions should match training data



Regularization

$$L = \frac{1}{N}\sum_{i=1}^{N} L_i(x_i; \boldsymbol{W}), y_i) \quad +\lambda R(\boldsymbol{W})$$

Why does this term work?

Data loss: Model predictions should match training data

Regularization: preference over the model.

Example:

L_2 regularization: $R(W) = ||W||_2 = \sum_k W_k^2$

L_1 regularization: $R(W) = ||W||_1 = \sum_k |W_k|$



Regularization

$$\text{L}_2 \text{ regularization: } R(W) = \left|\left|W\right|\right|_2 = \sum_k W_k^2$$
$$L_1 \text{ regularization: } R(W) = \left|\left|W\right|\right|_1 = \sum_k |W_k|$$

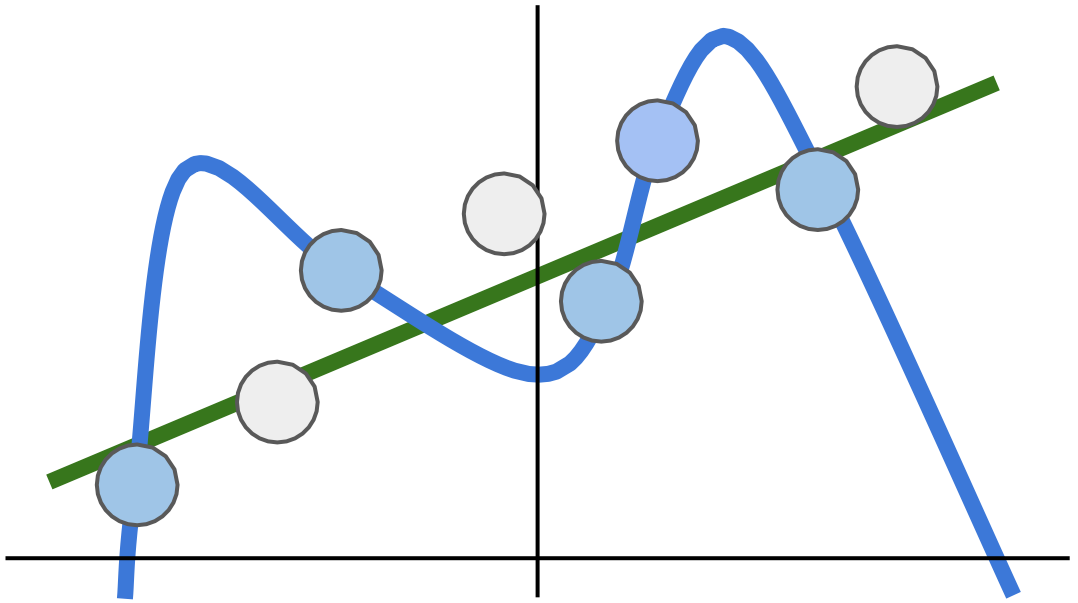
1. The "Maximum Margin" Principle
In an SVM, the goal is to find a decision boundary that separates classes.
Without Regularization: The model might try too hard to correctly classify every single noise point in the training data, leading to a very complex boundary.
With Regularization: Minimizing $\left|\left|W\right|\right|_2$ is mathematically equivalent to **maximizing the margin.** A larger margin usually leads to better performance on new, unseen data because it's more robust.

2. Controlling the "Weights"
Why small weights? If a weight W_i is very large, even a tiny change in a specific input feature x_i will cause a huge swing in the output. By forcing the weights to be small, regularization ensures that the model doesn't over-rely on any single feature or specific noise in the training set. This leads to a **smoother** model.



Regularization

If two models explain the data equally well, the simpler one is usually the correct one!



Occam's Razar: Among multiple competing hypotheses, the simplest is the best, William of Ockham 1285-1347



Other more complicated regularizations

- Dropout
  - Srivastava, Nitish, et al. “Dropout: a simple way to prevent neural networks from overfitting.” JMLR, 2014
- Batch normalization
  - Ioffe, Sergey, and Christian Szegedy. “Batch normalization: Accelerating deep network training by reducing internal covariate shift.” ICML, 2015.
- Mixup
  - Zhang, Hongyi, et al. “mixup: Beyond Empirical Risk Minimization.” *ICLR* 2018.

…



The scores of the SVM classifier is not probabilistically interpretable.

Solution: Softmax classifier!



Converting prediction score to probability



Probabilities must be >= 0

Probabilities must sum to 1

		exp		normalize	
cat	**3.2**	→	**24.5**	→	**0.13**
car	5.1	→	164.0	→	0.87
frog	-1.7	→	0.18	→	0.00

$s = f(x_i, W)$ $\quad e^s$ $\quad \frac{e^{s_k}}{\sum_j e^{s_j}}$

Softmax function



Loss function

Comparing two probability distributions:
Cross entropy

$$L = -\frac{1}{N}\sum_i p_i \log(q_i) = -\frac{1}{N}\sum_i \log\left(\frac{e^{s_{y_i}}}{\sum_j e^{s_j}}\right)$$



Probabilities must be >= 0

Probabilities must sum to 1

cat	**3.2**	exp →	**24.5**	normalize →	**0.13**	**1**
car	5.1		164.0		0.87	**0**
frog	-1.7		0.18		0.00	0

compare

$s = f(x_i, W)$ $\quad e^s$ $\quad \frac{e^{s_k}}{\sum_j e^{s_j}}$



Loss function

Comparing two probability distributions:
Cross entropy

$$L = -\frac{1}{N}\sum_i p_i \log(q_i) = -\frac{1}{N}\sum_i \log\left(\frac{e^{s_{y_i}}}{\sum_j e^{s_j}}\right)$$

Q1: What is the min/max possible cross entropy loss value?

A: When $s_{y_i} \to \infty$, i.e., all predictions are correct, then

$$L \approx -\frac{1}{N}\sum_i \log(1) = 0$$

When $s_{y_i} \to 0$, i.e., all predictions are wrong, then

$$L \approx -\frac{1}{N}\sum_i \log(0) \to \infty$$

Loss function

Comparing two probability distributions:
Cross entropy

$$L = -\frac{1}{N}\sum_i p_i \log(q_i) = -\frac{1}{N}\sum_i \log\left(\frac{e^{s_{y_i}}}{\sum_j e^{s_j}}\right)$$

Q2: At initialization all s will be approximately equal; what is the loss?

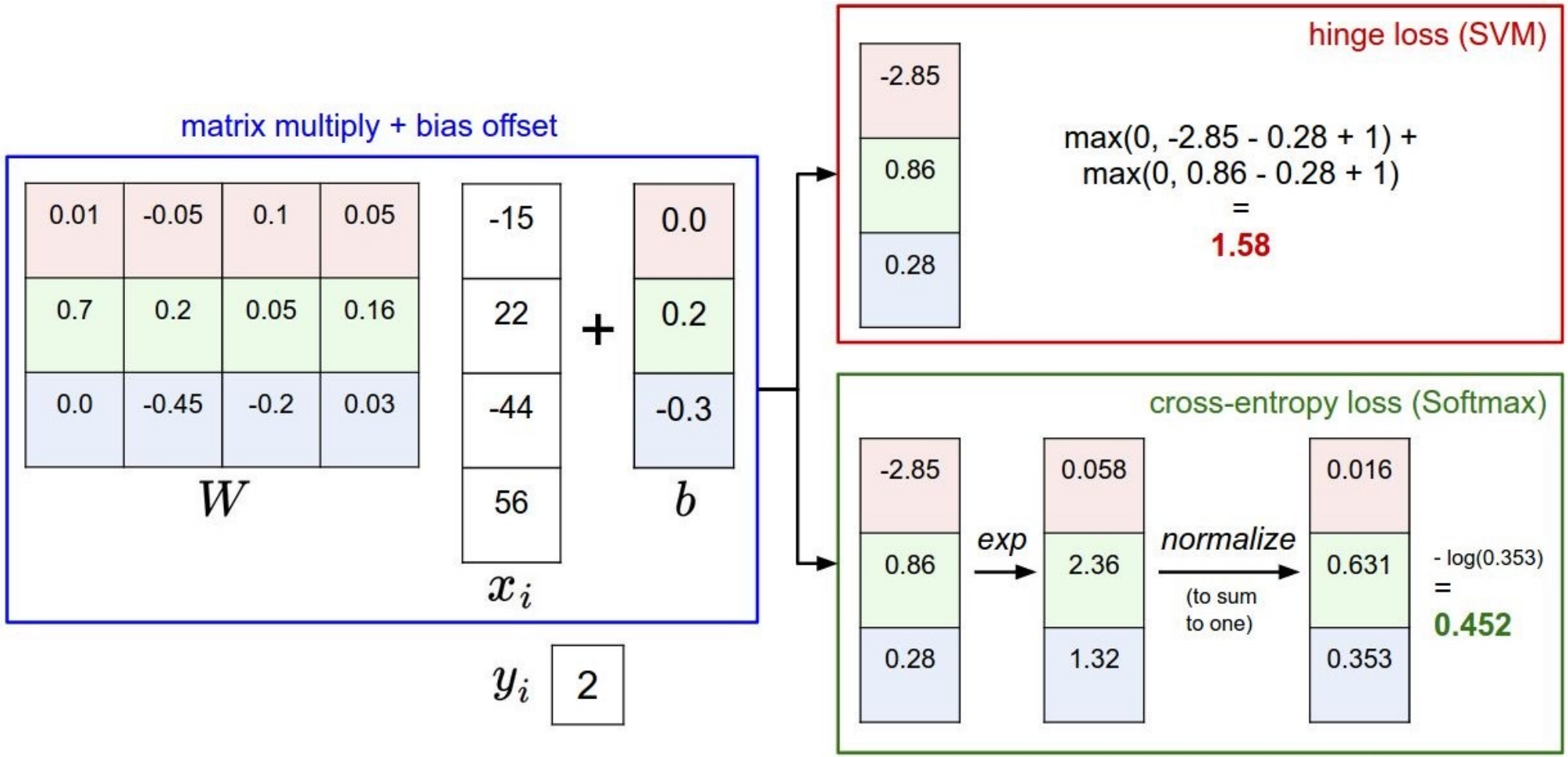
A: Assume there is C classes,

$$L = -\frac{1}{N}\sum_i \log\left(\frac{1}{C}\right)$$

If C=10, then $L = \log(10) \approx 2.3$.

Keep these values in mind when you are debugging your model.

Comparing Softmax classifier vs SVM



A more typical framework

Linear classifier

```
def train(images, labels):
  # Machine learning!
  return model
```



Define a model $f_W(x)$
Define a loss function $L(f_W(x), y)$
Solve W^* that leads to minimal L

```
def predict(model, test_images):
  # Use model to predict labels
  return test_labels
```



Predict the labels based on $f_{W^*}(x)$



Random search?

```
# assume X_train is the data where each column is an example (e.g. 3073 x 50,000)
# assume Y_train are the labels (e.g. 1D array of 50,000)
# assume the function L evaluates the loss function

bestloss = float("inf") # Python assigns the highest possible float value
for num in xrange(1000):
  W = np.random.randn(10, 3073) * 0.0001 # generate random parameters
  loss = L(X_train, Y_train, W) # get the loss over the entire training set
  if loss < bestloss: # keep track of the best solution
    bestloss = loss
    bestW = W
  print 'in attempt %d the loss was %f, best %f' % (num, loss, bestloss)

# prints:
# in attempt 0 the loss was 9.401632, best 9.401632
# in attempt 1 the loss was 8.959668, best 8.959668
# in attempt 2 the loss was 9.044034, best 8.959668
# in attempt 3 the loss was 9.278948, best 8.959668
# in attempt 4 the loss was 8.857370, best 8.857370
# in attempt 5 the loss was 8.943151, best 8.857370
# in attempt 6 the loss was 8.605604, best 8.605604
# ... (trunctated: continues for 1000 lines)
```



Random search?

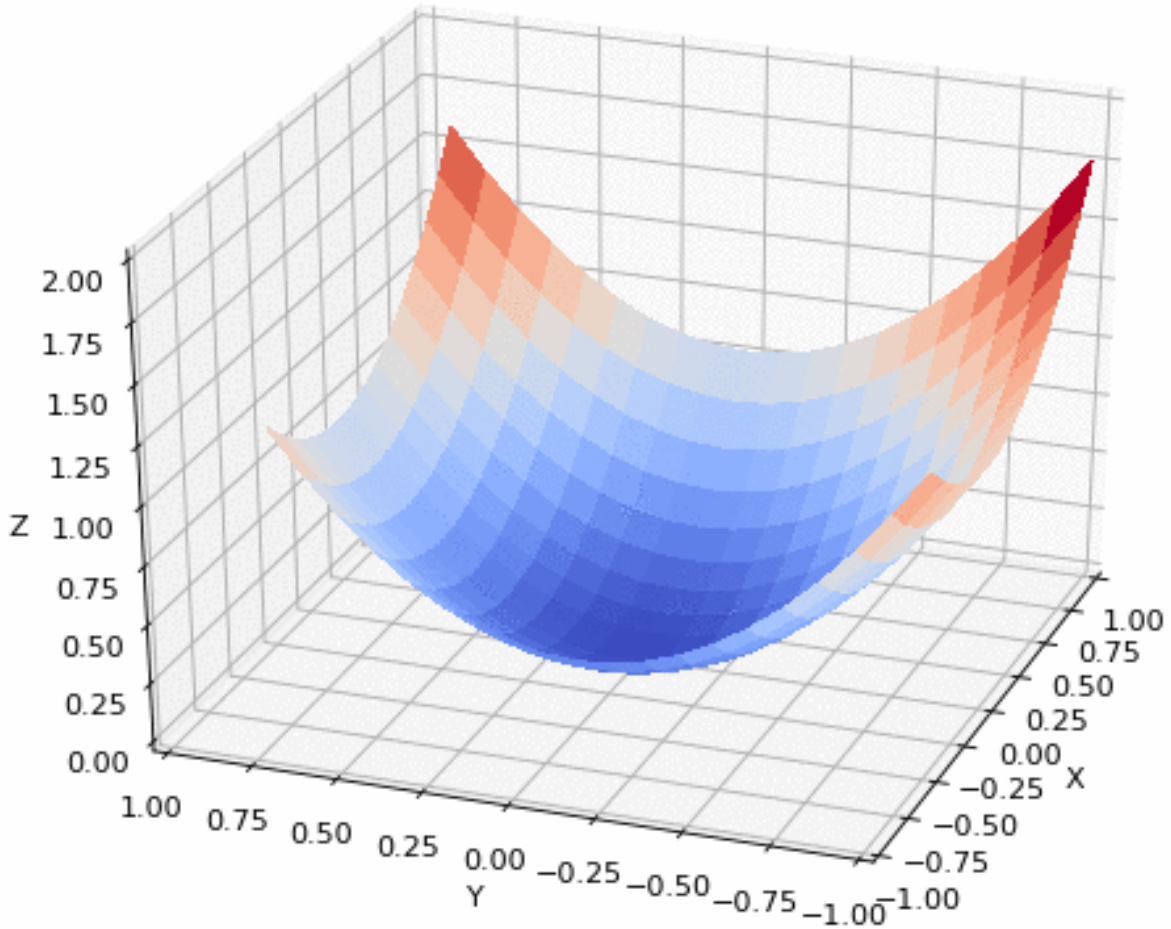
```
# Assume X_test is [3073 x 10000], Y_test [10000 x 1]
scores = Wbest.dot(Xte_cols) # 10 x 10000, the class scores for all test examples
# find the index with max score in each column (the predicted class)
Yte_predict = np.argmax(scores, axis = 0)
# and calculate accuracy (fraction of predictions that are correct)
np.mean(Yte_predict == Yte)
# returns 0.1555
```

15.5% accuracy (SOTA is ~99.7%)

Random search is really slow



Gradient descent



Idea: follow the direction that leads to steepest loss reduction.

$$W \leftarrow W - \lambda \frac{\partial L}{\partial W}$$

Step size

Gradient

Gradient descent

$$\boldsymbol{W} \leftarrow \boldsymbol{W} - \lambda \frac{\partial L}{\partial \boldsymbol{W}} = \boldsymbol{W} - \lambda \frac{\partial \frac{1}{N} \sum_{i=1}^{N} L_i}{\partial \boldsymbol{W}} = \boldsymbol{W} - \lambda \frac{1}{N} \sum_{i=1}^{N} \frac{\partial L_i}{\partial \boldsymbol{W}}$$

$$L_i = \sum_{j \neq y_i} \max\left(0, \boldsymbol{W}_j^T x_i - \boldsymbol{W}_{y_i}^T x_i + 1\right)$$



Gradient descent

$$L_i = \sum_{j \neq y_i} \max\left(0, \boldsymbol{W}_j^T x_i - \boldsymbol{W}_{y_i}^T x_i + 1\right)$$

When $\boldsymbol{W}_j^T x_i - \boldsymbol{W}_{y_i}^T x_i + 1 \leq 0$, then $L_i = 0$, and the gradient is 0

When $\boldsymbol{W}_j^T x_i - \boldsymbol{W}_{y_i}^T x_i + 1 > 0$, then

$$\frac{\partial L_i}{\partial \boldsymbol{W}_{y_i}} = -x_i \qquad \frac{\partial L_i}{\partial \boldsymbol{W}_j} = x_i$$



Loss Function

- We must define a measure of error.
- How wrong is our model?

- Mean square error
  - Find the difference between the actual y and predicted y value ($y = mx + c$), for a given x.
  - Square this difference.
  - Find the mean of the squares for every value in X.

$$E = \frac{1}{n}\sum_{i=0}^{n}(y_i - (mx_i + c))^2$$



Linear Regression

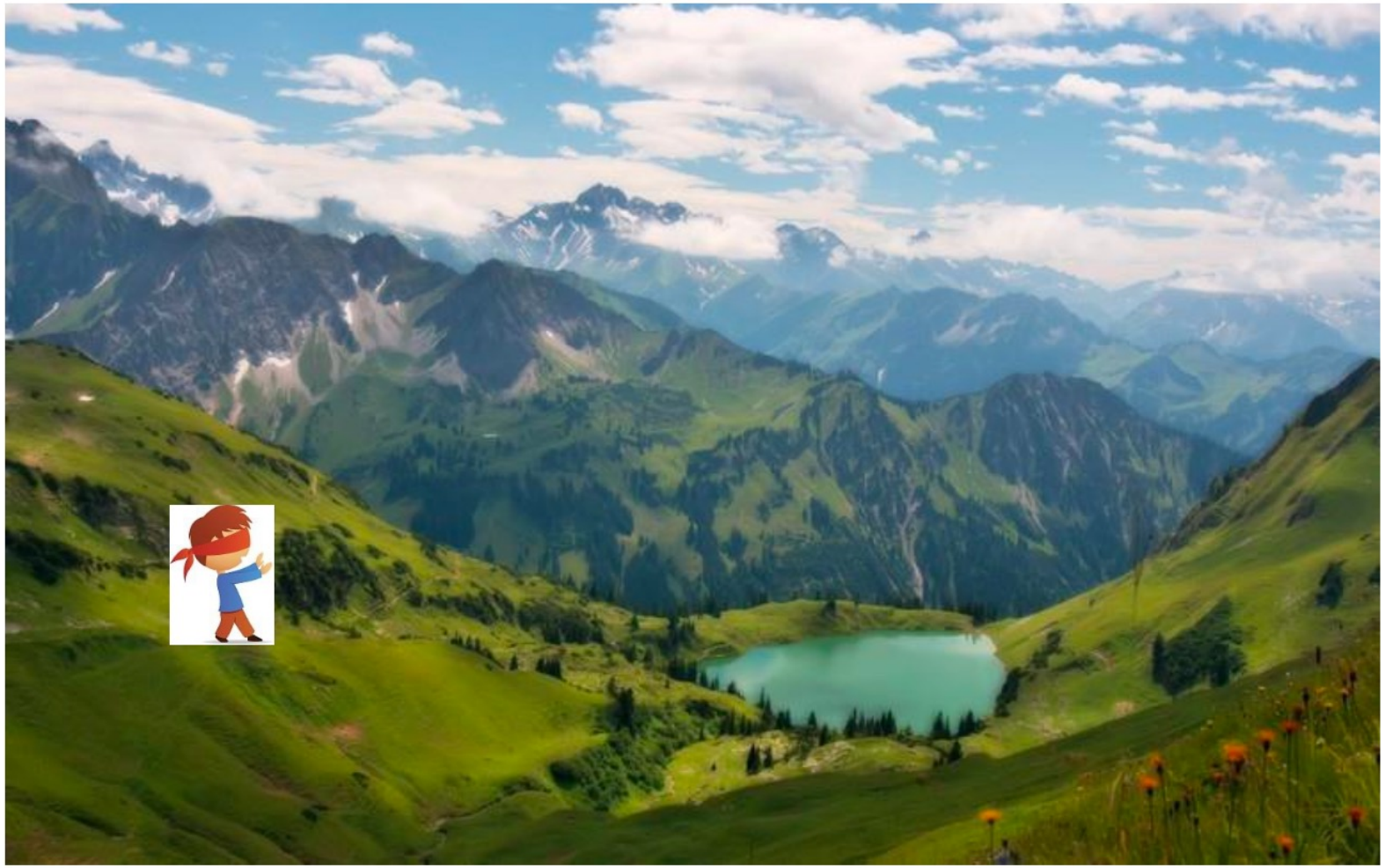
- One central tenet of supervised learning

Find the model parameters that lead to smallest error on all possible data.

- We only observe limited amount of data, so it is usually taken as minimizing error on training data (empirical error).



Optimization

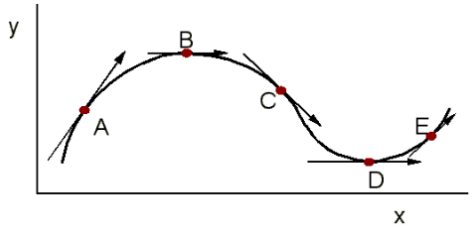


What is a gradient?

- Geometrically
  - Points in the direction of the greatest rate of increase of the function and its magnitude is the slope of the graph in that direction

- More formally in 1D

$$\frac{\partial f(x)}{\partial x} = \lim_{h \to 0} \frac{f(x+h) - f(x)}{h}$$



- In higher dimensions

$$\frac{\partial f}{\partial x_i}(a_1, \dots, a_n) = \lim_{h \to 0} \frac{f(a_1, \dots, a_i + h, \dots, a_n) - f(a_1, \dots, a_i, \dots, a_n)}{h}$$



Numeric gradient

- Can set h to a very low number and compute:

$$\frac{\partial f(x)}{\partial x} = \frac{f(x+h)-f(x)}{h}$$

- Slow and just an approximation
  - Need to compute score once for each parameter
  - Sensitive to choice of h

- h needs to be chosen as well - hyperparameter

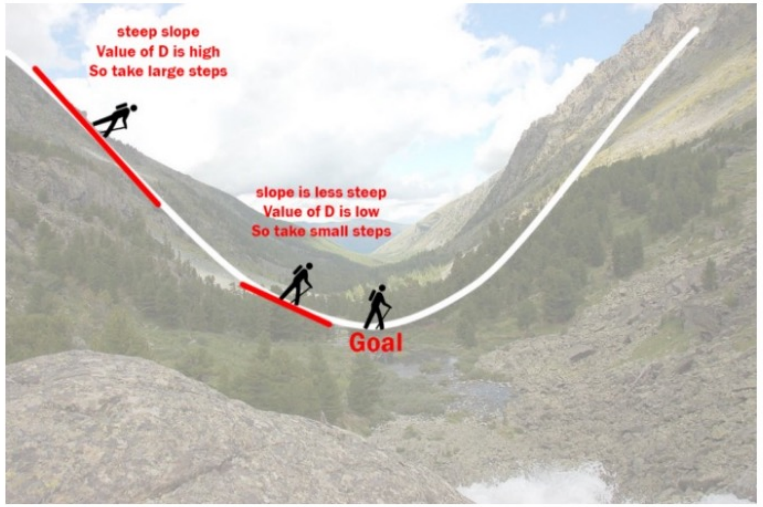


Gradient Descent

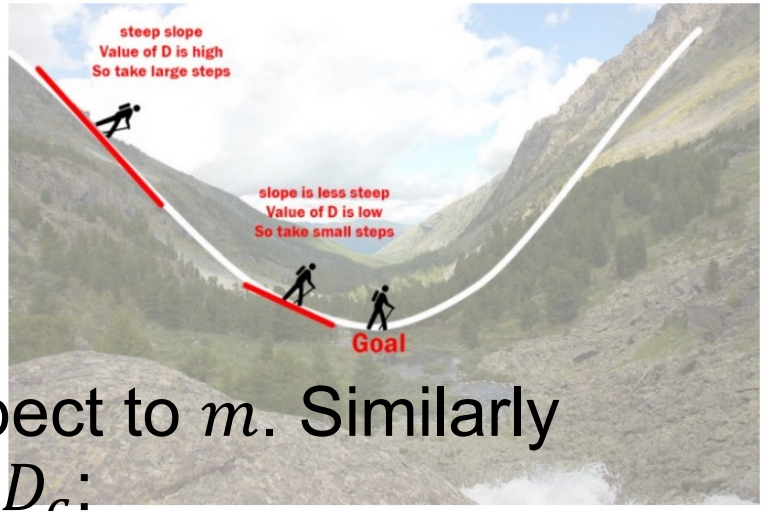
1. Initially let $m = 0$ and $c = 0$. Let L be our learning rate. This controls how much the value of m changes with each step. L could be a small value like 0.001 for good accuracy.
2. Calculate the partial derivative of the loss function with respect to m, and plug in the current values of x, y, m and c in it to obtain the derivative value $\boldsymbol{D}$.

$$D_m = \frac{1}{n} \sum_{i=0}^{n} 2(y_i - (mx_i + c))(-x_i)$$

$$D_m = \frac{-2}{n} \sum_{i=0}^{n} x_i(y_i - \bar{y}_i)$$



Gradient Descent



3. D_m is the value of the partial derivative with respect to m. Similarly let's find the partial derivative with respect to c, D_c:

$$D_c = \frac{-2}{n}\sum_{i=0}^{n}(y_i - \bar{y}_i)$$

4. Now we update the current value of m and c using the following equation:

$$m = m - L \times D_m$$

$$c = c - L \times D_c$$

5. We repeat this process until our loss function is a very small value or ideally 0 (which means 0 error or 100% accuracy). The value of m and c that we are left with now will be the optimum values.



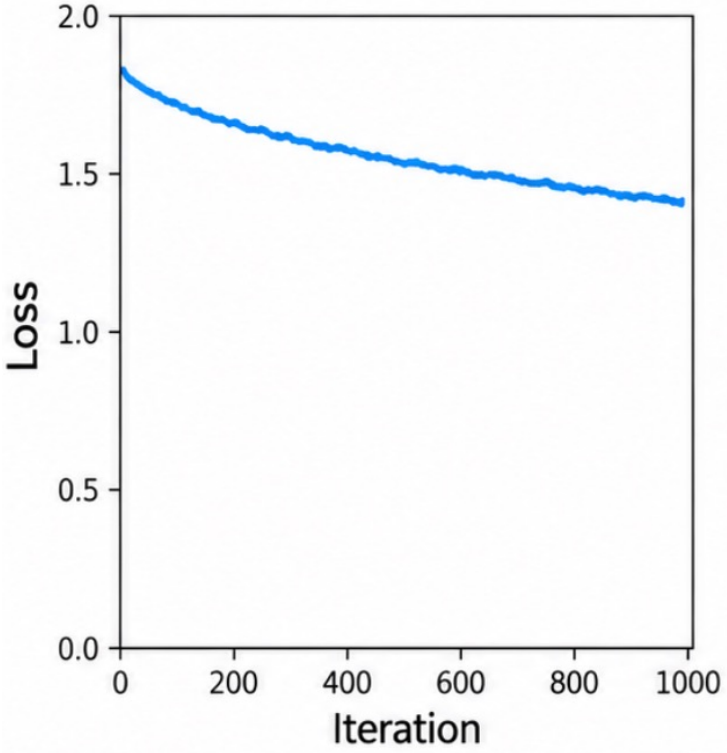
Iterative Gradient Descent

- Run forward to compute the loss
- Run backward to compute the gradients
- Average the loss and the gradient for each parameter vector
- At each iteration, update each parameter vector in the direction opposite to its gradient so the loss decreases

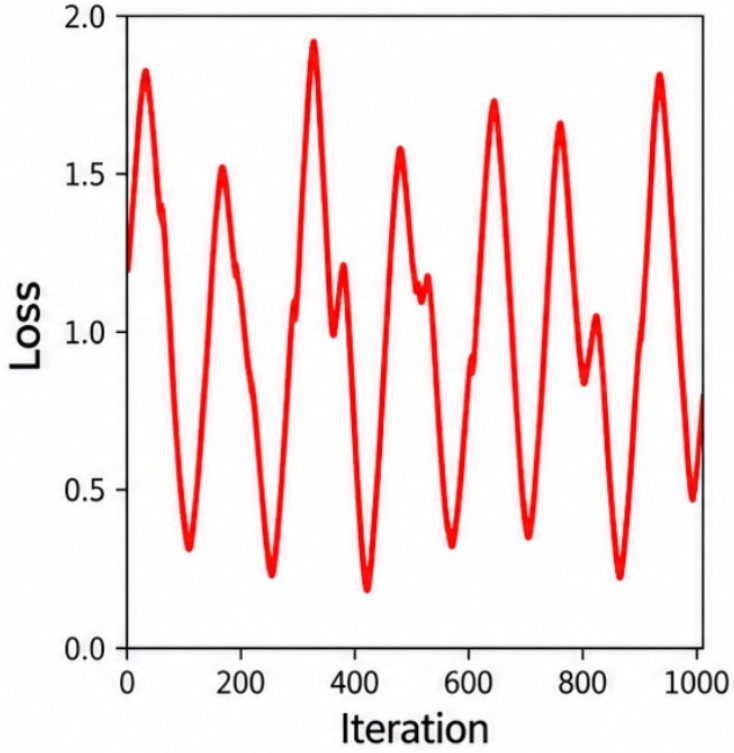


Diagnose the Training

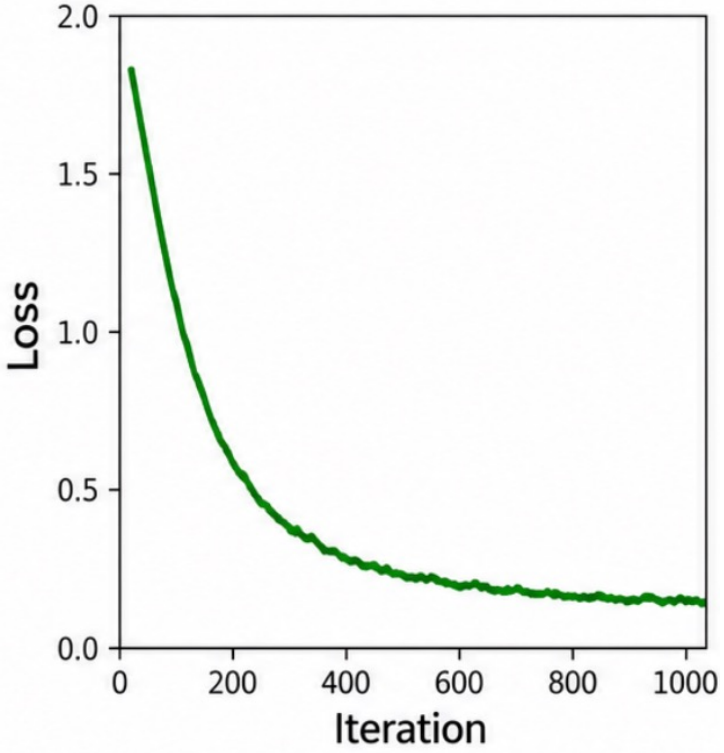
What's wrong?



Learning rate may be too small



Learning rate may be too large



Looks reasonable



Gradient descent

A more general solution?

Autograd!



Computation Graph

Assume $z = (\langle \mathbf{x}, \mathbf{w} \rangle - y)^2$

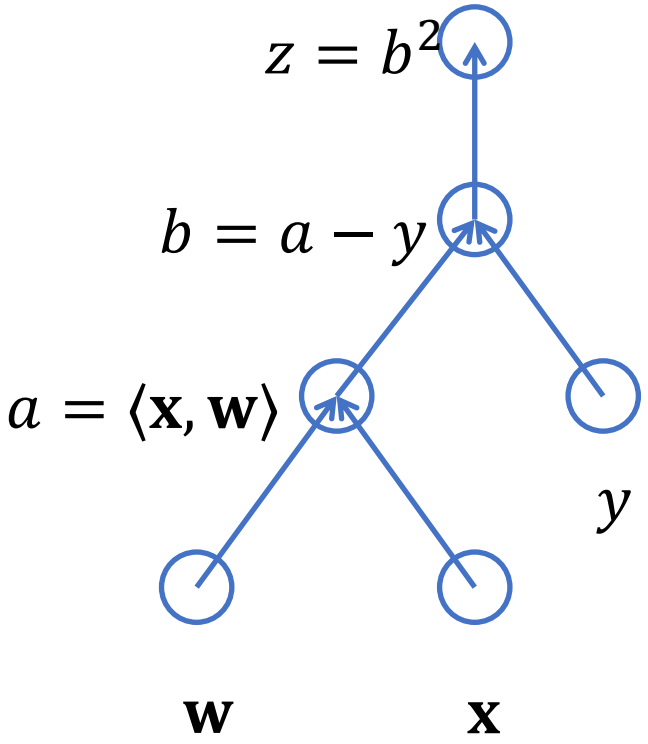
Forward

- Decompose into primitive operations
- Build a directed acyclic graph to present the computation

Backward

- Get the gradient by chain rule

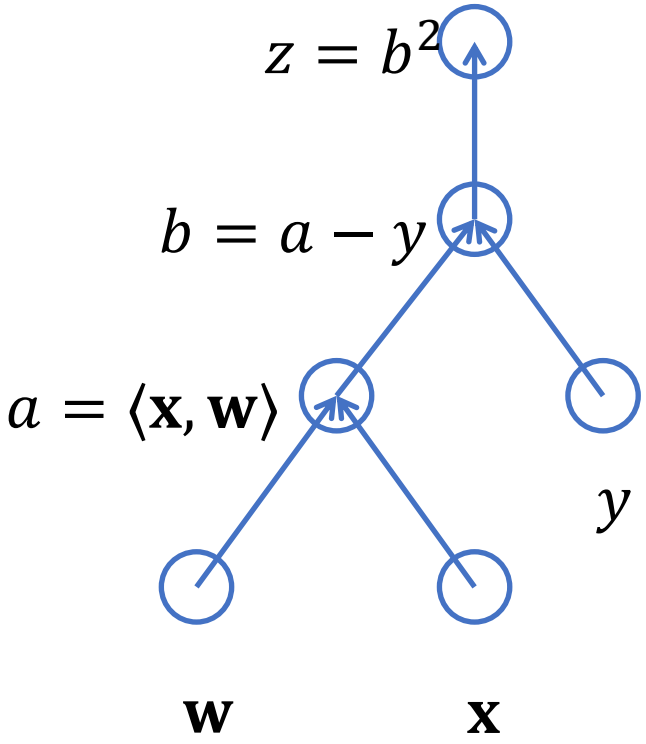
$$\frac{\partial y}{\partial x} = \frac{\partial y}{\partial u_n} \frac{\partial u_n}{\partial u_{n-1}} \cdots \frac{\partial u_2}{\partial u_1} \frac{\partial u_1}{\partial x}$$



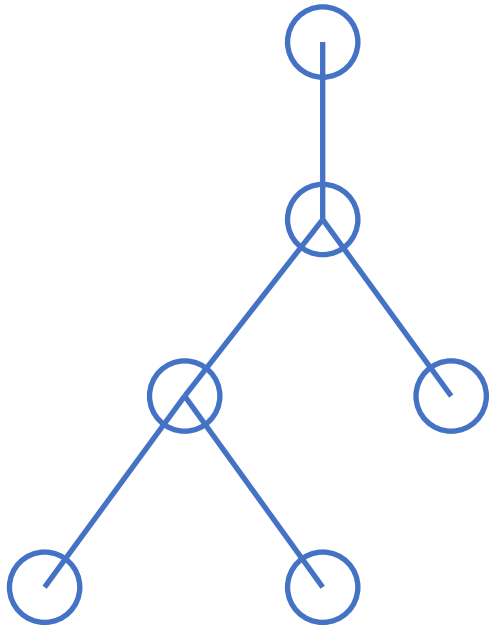
Reverse Accumulation

Assume $z = (\langle \mathbf{x}, \mathbf{w} \rangle - y)^2$

Forward



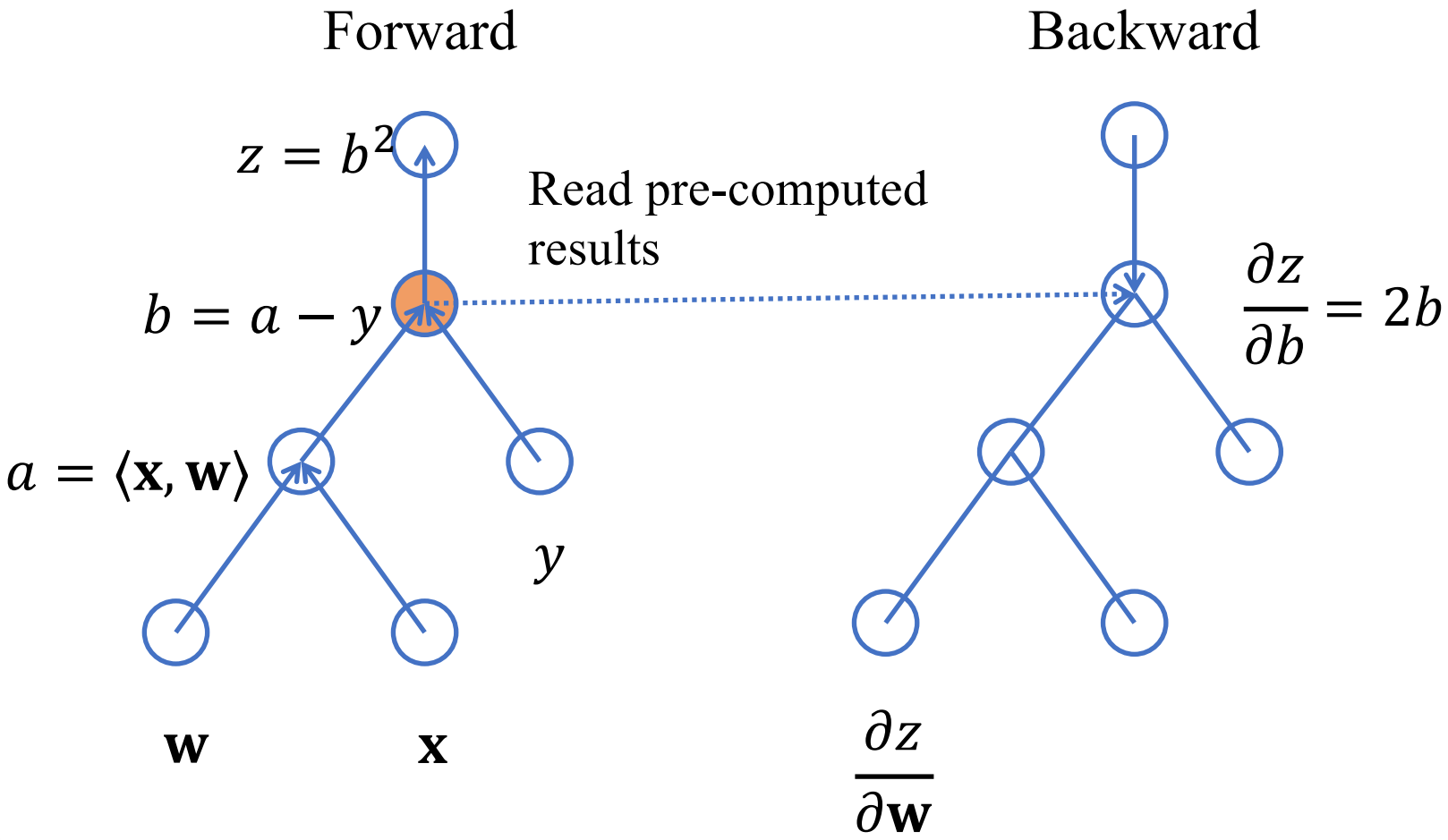
Backward



$\frac{\partial z}{\partial \mathbf{w}}$

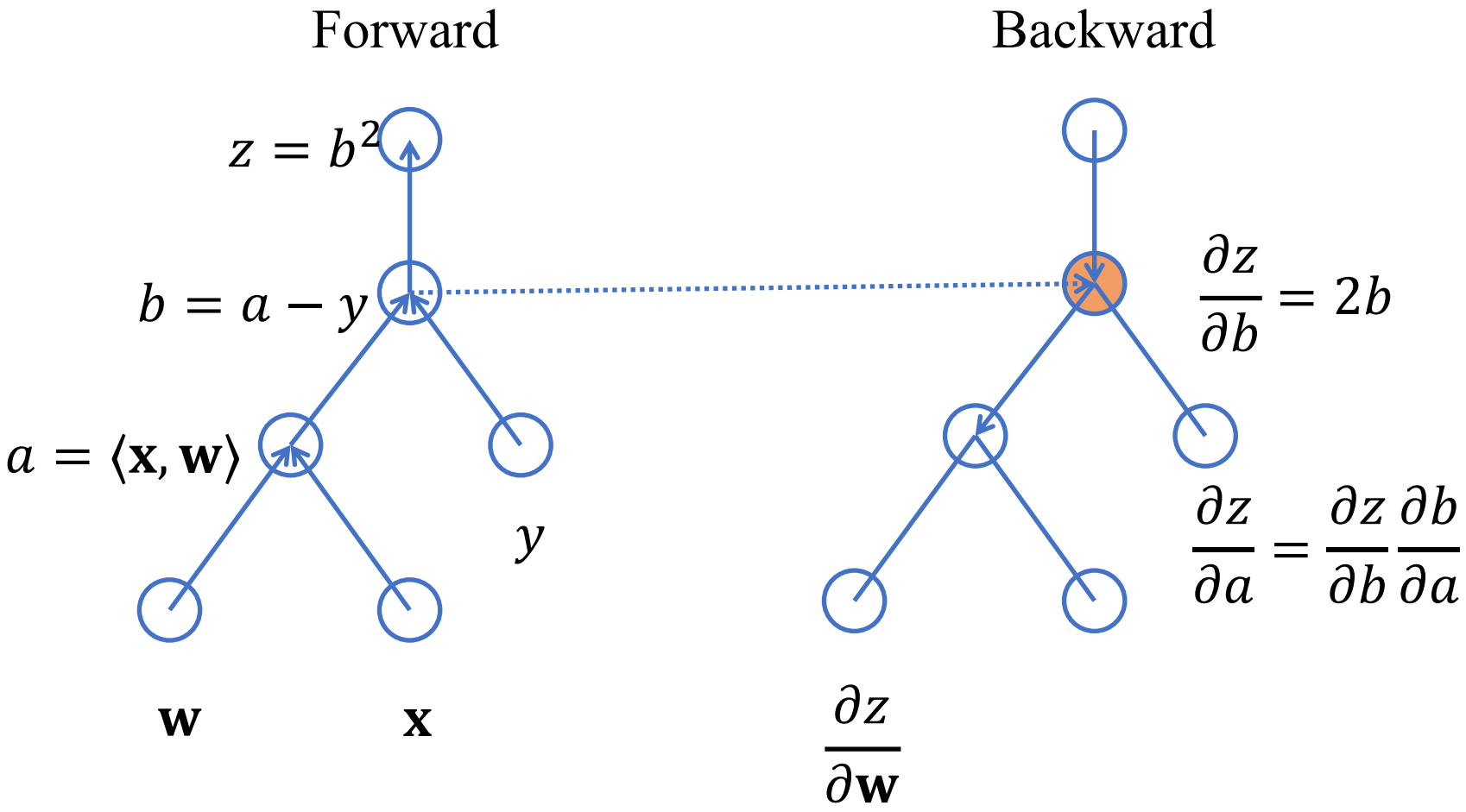
Reverse Accumulation

Assume $z = (\langle \mathbf{x}, \mathbf{w} \rangle - y)^2$



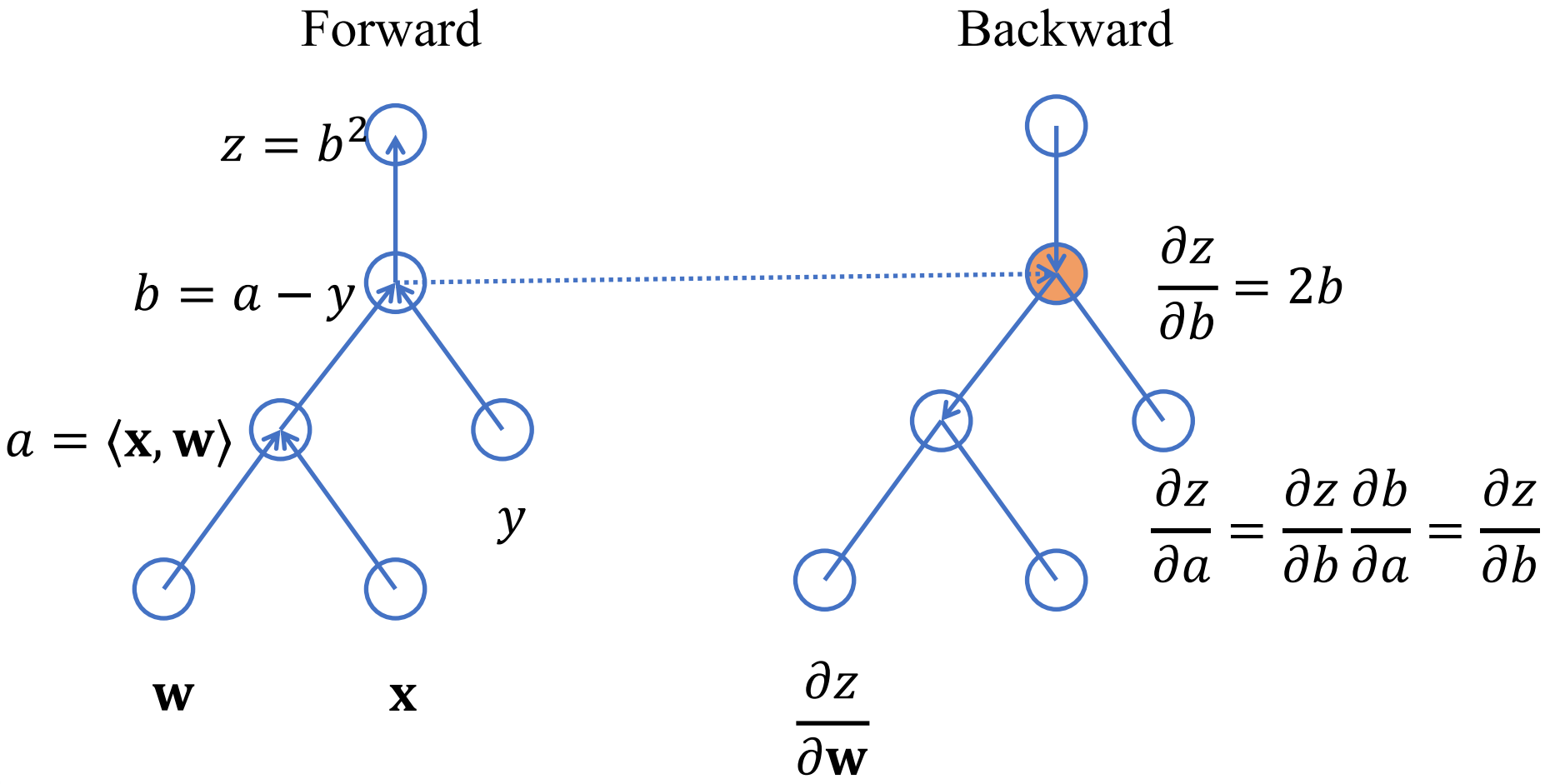
Reverse Accumulation

Assume $z = (\langle \mathbf{x}, \mathbf{w} \rangle - y)^2$



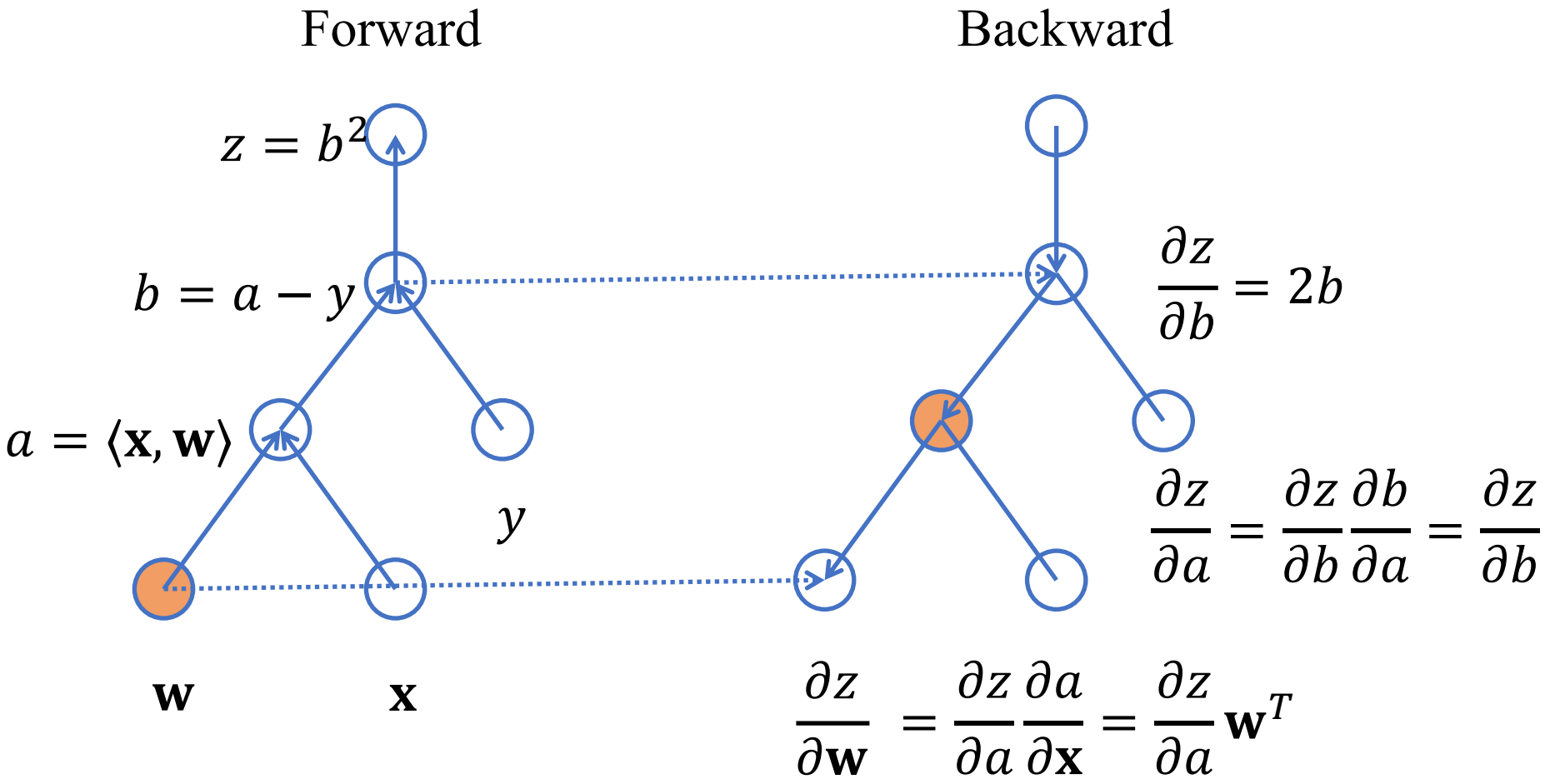
Reverse Accumulation

Assume $z = (\langle \mathbf{x}, \mathbf{w} \rangle - y)^2$



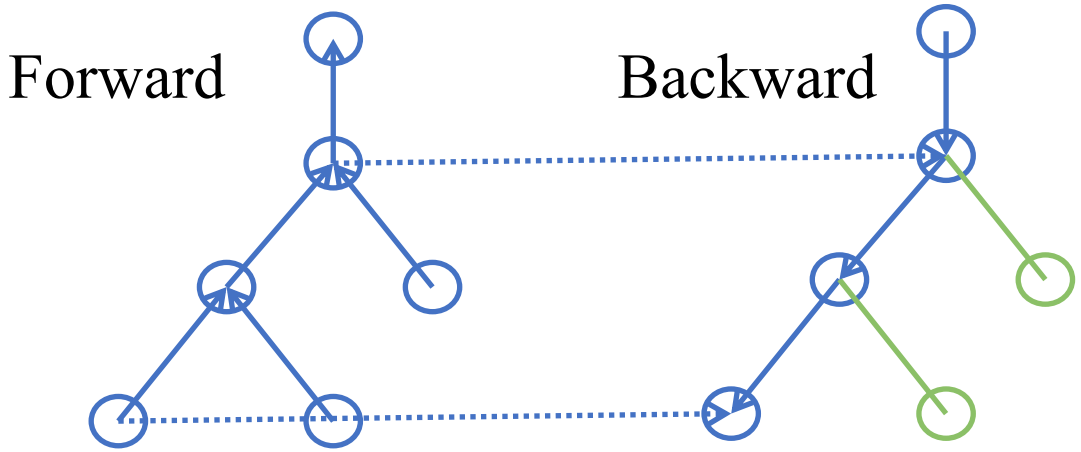
Reverse Accumulation

Assume $z = (\langle \mathbf{x}, \mathbf{w} \rangle - y)^2$



Reverse Accumulation Summary

- Build a computation graph
- Forward: Evaluate the graph, store intermediate results
- Backward: Evaluate the graph in a reversed order
  - Eliminate paths not needed





Stochastic Gradient descent

$$W \leftarrow W - \lambda \frac{\partial L}{\partial W} = W - \lambda \frac{\partial \frac{1}{N} \sum_{i=1}^{N} L_i}{\partial W}$$

Expensive when N is large



Use a batch of samples for approximation.
B can be chosen based on the memory/learning speed…

$$W \leftarrow W - \lambda \frac{\partial L}{\partial W} = W - \lambda \frac{\partial \frac{1}{B} \sum_{i=1}^{B} L_i}{\partial W}$$



Exit Quiz

